Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования «Санкт-Петербургский
национальный исследовательский университет информационных
технологий, механики и оптики»



Факультет «**программной инженерии и компьютерной техники**»

ОТЧЕТ

По Научно-исследовательской работе

по теме:

«Аппаратная реализация сверточных нейронных сетей на ПЛИС»

Обучающийся:	Альосман Мохаммад Кусае
Группа:	P4119
Руководитель:	Быковский Сергей Вячеславович
Руководитель практики от университета:	Маркина Татьяна Анатольевна

Санкт-Петербург
2026

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

Таблица 1 – Перечень сокращений и обозначений

Сокращение	**Расшифровка**
CNN	Convolutional Neural Network, сверточная нейронная сеть
FPGA / ПЛИС	Field-Programmable Gate Array, программируемая логическая интегральная схема
HLS	High-Level Synthesis, высокоуровневый синтез
RTL	Register Transfer Level, уровень регистровых передач
LUT	Look-Up Table, базовый логический ресурс ПЛИС
BRAM	Block RAM, блочная память ПЛИС
DSP	Digital Signal Processing block, аппаратный арифметический блок
UDM	Отладочный memory-mapped интерфейс, используемый для доступа к регистрам и памяти проекта
INT8	8-разрядное целочисленное представление данных и весов
MAC	Multiply-Accumulate, операция умножения с накоплением
PE	Processing Element, элемент регулярного вычислительного массива
WNS	Worst Negative Slack, минимальный запас времени по setup
TNS	Total Negative Slack, суммарный отрицательный slack
ALM	Adaptive Logic Module, логический ресурс Intel/Altera FPGA
QNN / BNN	Quantized Neural Network / Binarized Neural Network

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ **1**

ВВЕДЕНИЕ **4**

1 Обзор и выбор открытой реализации для CNN на ПЛИС **9**

1.1 Критерии сравнения 9

1.2 Рассмотренные решения 9

1.3 Сводная сравнительная таблица 11

1.4 Преимущества, ограничения и требования к плате 13

1.5 Выбор направления дальнейшей реализации 15

2 Запуск выбранного проекта на отладочной плате ПЛИС и сборка **16**

2.1 Подготовка контрольной CNN-конфигурации 16

2.1.1 Структура вычислительного графа 16

2.1.2 Типы данных и программная структура 17

2.2 Процесс проверки контрольной реализации 18

2.2.1 C-симуляция HLS-модели 18

2.2.2 HLS-синтез контрольной конфигурации 18

2.2.3 RTL-интеграция и memory-mapped интерфейс 19

2.2.4 RTL-симуляция интегрированного проекта 19

2.2.5 Синтез, implementation, timing и power analysis 20

2.2.6 Аппаратная проверка на FPGA 21

2.3 Архитектурная доработка сверточного ядра 22

2.3.1 Мотивация доработки 22

2.3.2 Структура line buffer и MAC-ядра 22

2.3.3 Проверка оптимизированной версии 23

2.4 Сопоставление полученных результатов 25

2.4.1 Сравнение контрольной и оптимизированной реализаций . . . 25

2.5 Итоги выполненных работ в ходе практики 26

2.6 Направления дальнейшей работы 28

ЗАКЛЮЧЕНИЕ **29**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **31**

ПРИЛОЖЕНИЕ А. План-график выполнения индивидуального задания 32

ПРИЛОЖЕНИЕ Б. Подробные сведения по сравнению открытых решений 33

ПРИЛОЖЕНИЕ В. Материалы HLS и RTL-проверки контрольной версии 34

ПРИЛОЖЕНИЕ Г. Материалы проверки оптимизированной версии 41

ПРИЛОЖЕНИЕ Д. Фрагменты исходного кода 46

ВВЕДЕНИЕ

Практика была посвящена теме аппаратной реализации сверточных нейронных сетей на ПЛИС. Актуальность темы связана с тем, что FPGA-устройства позволяют создавать специализированные вычислительные тракты для inference-задач, обеспечивая контролируемую латентность, возможность аппаратного параллелизма и умеренное энергопотребление.

Исходная формулировка индивидуального задания требовала не только получить результаты запуска собственной реализации, но и предварительно оценить открытые реализации CNN/нейронных ускорителей на ПЛИС. Поэтому работа была разделена на две взаимосвязанные части. Первая часть включала обзор открытых проектов и инструментальных решений с оценкой поддерживаемых моделей и слоев, ресурсов LUT/BRAM/DSP, частоты, latency/throughput, ограничений и требований к плате. Вторая часть была посвящена практическому маршруту реализации: HLS-модель, HLS-синтез, RTL-интеграция, Vivado implementation и аппаратная проверка на FPGA.

В отчете последовательно рассматриваются этапы выполнения практики: обзор и сравнение открытых решений для реализации CNN на ПЛИС, выбор направления дальнейшей разработки, подготовка инструментальной среды, реализация CNN-ускорителя, его проверка на уровнях HLS, RTL и FPGA, а также экспериментальная архитектурная доработка сверточного ядра. Подробные экспериментальные материалы, включающие расширенные таблицы, скриншоты отчетов HLS/Vivado и фрагменты исходного кода, приведены в приложениях.

Тема, цель и задачи практики.

Тема практики связана с аппаратной реализацией сверточных нейронных сетей на программируемых логических интегральных схемах. Работа была направлена на изучение открытых подходов к реализации CNN-вычислений на FPGA, выбор реализуемого направления дальнейшей разработки, а также выполнение полного практического цикла проверки выбранной реализации: от анализа архитектурных решений до запуска прототипа на отладочной плате.

Целью практики являлось получение практических навыков анализа, проектирования, синтеза и аппаратной проверки компактного CNN-ускорителя на FPGA, а также оценка его характеристик с точки зрения поддерживаемых операций, использования ресурсов, временных ограничений, производительности и

пригодности для дальнейших архитектурных экспериментов.

Для достижения указанной цели были поставлены следующие задачи:

- изучить открытые реализации и инструментальные экосистемы для выполнения CNN-вычислений на FPGA;
- сравнить рассмотренные решения по поддерживаемым моделям и слоям, используемым ресурсам, частоте, производительности, ограничениям и требованиям к аппаратной платформе;
- выбрать реализуемое направление дальнейшей разработки и обосновать выбор с учетом доступности инструментов, сложности воспроизведения и возможности последующего анализа;
- подготовить инструментальную среду для HLS- и RTL-проектирования;
- реализовать компактную inference-ориентированную CNN-конфигурацию, пригодную для HLS-синтеза;
- выполнить функциональную проверку модели на уровне C-симуляции;
- провести HLS-синтез и получить предварительную оценку ресурсов и латентности;
- интегрировать полученный RTL-модуль в проект верхнего уровня для отладочной платы FPGA;
- выполнить RTL-симуляцию интегрированного проекта и проверить корректность memory-mapped взаимодействия;
- провести synthesis, implementation, timing analysis и оценку энергопотребления в Vivado;
- выполнить аппаратную проверку реализации на FPGA;
- разработать и проверить экспериментальную архитектурную доработку сверточного ядра;
- сопоставить результаты контрольной и оптимизированной реализаций и определить ограничения выполненного эксперимента.

В последующих разделах отчета указанные задачи рассматриваются в логической последовательности: сначала приводится обзор и сравнение открытых решений, затем обосновывается выбор реализуемого направления, после чего описываются этапы реализации, проверки, аппаратного запуска и архитектурной оптимизации.

Логика выполнения этапов.

Работа выполнялась поэтапно. Сначала была собрана аналитическая база для сравнения открытых решений. Затем был выбран практический путь, который не требует воспроизведения крупной промышленной accelerator-платформы, но позволяет построить проверяемый и модифицируемый CNN-ускоритель. После этого была реализована контрольная HLS-конфигурация, которая стала контрольной конфигурацией для дальнейшей оптимизации. Завершающая часть практики включала разработку версии с line buffer, ее синтез, интеграцию и аппаратную проверку.

Таблица 2 – План-график выполнения работ в рамках практики

№ этапа	**Название этапа**	**Задание этапа**
1	Инструктаж обучающегося	Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка.
2	Обзор и выбор открытой реализации для CNN на ПЛИС	Рассмотреть открытые реализации для выполнения вычислений CNN на ПЛИС. Сравнить решения по характеристикам: поддерживаемые модели/слои, требуемые ресурсы (LUT/BRAM/DSP), частота, производительность (latency/throughput), ограничения и требования. Составить таблицу сравнения, описать преимущества/недостатки и выбрать наиболее подходящее решение для дальнейших экспериментов (с аргументацией выбора). Оценить требуемые ресурсы для подбора отладочной платы.

№ этапа	Название этапа	Задание этапа
3	Запуск выбранного проекта на отладочной плате ПЛИС и сборка	Подготовить окружение разработки, собрать/синтезировать проект, устранить ошибки сборки. Запустить выбранную реализацию на отладочной плате (подобранной по ресурсам), проверить корректность работы на тестовых данных. Зафиксировать конфигурацию (частота, параметры, ограничения) и результаты первичного запуска. Провести измерения производительности на выбранном приложении (например, обработка изображений / inference CNN): latency, throughput, использование ресурсов (LUT/BRAM/DSP), при возможности — энергопотребление. Сравнить результаты с контрольной конфигурацией (например, значения из статьи/других реализаций или вариант без оптимизаций). Подготовить выводы по преимуществам и недостаткам.
4	Оформление отчётных документов в соответствии с требованиями	1. Должно быть подробное описание выполнения задач по этапам. Результаты задания необходимо разместить в приложения. 2. Оформление отчёта должно быть выполнено в соответствии с методическим пособием (https://books.ifmo.ru/file/pdf/2622.pdf) 3. Структура документа: титульный лист, введение, основная часть, заключение, примечания. 4. В основной части подробно описывается выполнение задач всех этапов, исключая обязательные.

№ этапа	**Название этапа**	**Задание этапа**
5	Получить отзыв руководителя практики	Получить отзыв с оценкой у руководителя практики в модуле Практики

1 Обзор и выбор открытой реализации для CNN на ПЛИС

1.1 Критерии сравнения

Сравнение выполнялось по тем характеристикам, которые необходимы для выбора дальнейшего направления практической реализации: поддерживаемые модели и слои, требуемые ресурсы, частота, производительность, ограничения, требования к плате, доступность исходного кода и пригодность для модификации.

При анализе важно разделять три типа данных. Первый тип - опубликованные отчеты synthesis/implementation, которые можно прямо использовать как экспериментальные результаты. Второй тип - результаты из статей, которые относятся к конкретной модели, FPGA и конфигурации. Третий тип - предварительные оценки по RTL, допустимые только при отсутствии готовых отчетов, но требующие явного указания статуса данных. Такой подход позволяет не подменять отсутствующие результаты предположениями.

1.2 Рассмотренные решения

Для выбора реализуемого направления я проанализировал несколько открытых проектов и инструментальных экосистем, связанных с аппаратной реализацией нейросетевых вычислений на FPGA. Эти решения различаются по уровню зрелости, целевым моделям, способу описания аппаратуры, поддерживаемым типам данных и полноте опубликованных экспериментальных результатов. Поэтому я рассматривал их не только как готовые проекты для возможного воспроизведения, но и как источники архитектурных идей, которые могли быть использованы при построении собственного практического прототипа.

- **Tiny TPU** - оказался для меня важным примером компактной TPU-подобной архитектуры. При его анализе я обратил внимание на модульное разделение ускорителя на вычислительное ядро, буферизацию, управляющую часть и блок постобработки. Именно эта идея помогла мне лучше понять, что даже небольшой accelerator должен рассматриваться не только как набор арифметических операций, но и как система из нескольких взаимосвязанных блоков. Поэтому из Tiny TPU для дальнейшей работы была важна не попытка полного воспроизведения проекта, а понимание верхнеуровневой декомпозиции аппаратного ускорителя.
- **FREE-TPU V3+ for FPGA** - я рассматривал как пример более современной inference-ориентированной accelerator-платформы. Для моей работы

он был важен тем, что показал необходимость учитывать пониженную точность вычислений, поддержку разных типов моделей и организацию dataflow. Этот проект помог мне сформировать вывод, что дальнейшая реализация не должна быть ориентирована на обучение нейронной сети, а должна рассматриваться именно как inference-ускоритель с компактным представлением данных. В то же время проект оказался слишком сложным для прямого воспроизведения в рамках данного этапа, поэтому я использовал его главным образом как архитектурный ориентир.

- **NPU_X_Interface** - был для меня особенно полезен с точки зрения понимания внутренней структуры convolution-пути. В нем явно выделяются стадии подготовки входных данных, загрузки весов, выполнения вычислений, накопления частичных сумм и записи результата. Благодаря этому проекту я лучше понял, что сверточный слой на FPGA желательно рассматривать не как одну монолитную функцию, а как последовательность аппаратных стадий. Эта идея затем была учтена при анализе собственной реализации и при разработке оптимизированного сверточного ядра с line buffer и частичными суммами.
- **FINN** - я рассматривал как пример зрелого подхода к построению quantized dataflow accelerators. Этот проект помог мне увидеть, что современные FPGA-ускорители нейронных сетей часто строятся не как универсальные процессоры, а как специализированные потоковые структуры под конкретную квантизованную модель. Для моей практической реализации FINN не был выбран как непосредственная база из-за сложности инструментального потока, однако он был важен для обоснования выбранного направления: low-precision inference, dataflow-ориентированность и аппаратная специализация под конкретную сеть.
- **hls4ml** - оказался важным примером того, что HLS-подход может использоваться как практический путь от модели машинного обучения к синтезируемому аппаратному описанию. Для моей работы это было особенно важно, поскольку практическая часть строилась именно вокруг HLS/RTL-маршрута: сначала C/C++-описание вычислительного ядра, затем HLS-синтез, интеграция RTL-модуля в Vivado-проект и аппаратная проверка на FPGA. Анализ hls4ml подтвердил, что выбранный мной путь через HLS является реалистичным для компактного исследовательского прототипа.

Таким образом, рассмотренные проекты помогли мне сформировать собственное реализуемое направление. Tiny TPU дал представление о модульной организации ускорителя, FREE-TPU V3+ — о важности inference-ориентированного low-precision подхода, NPU_X_Interface — о структуре convolution-пути и частичных суммах, FINN — о quantized dataflow accelerator как современном ориентире, а hls4ml — о практической применимости HLS-маршрута. На основе этого анализа было принято решение не копировать один из проектов полностью, а реализовать компактный CNN inference accelerator, который можно проверить на всех уровнях: HLS, RTL, Vivado implementation и реальная FPGA.

1.3 Сводная сравнительная таблица

Таблица 3 – Сравнение рассмотренных решений: модели, ресурсы и частота

Решение	**Поддерживаемые модели/слои**	**Ресурсы LUT/BRAM/DSP**	**Частота**
Tiny TPU	XOR/MLP, MNIST demo, matrix operations, systolic array, post-processing	MNIST JTAG: 7003 ALM, 101 RAM blocks, 7 DSP на Cyclone V	50 МГц demo clock
FREE-TPU V3+	CNN, LSTM, RNN; YOLO, ResNet, MobileNet; FP16/INT8	Точные FPGA LUT/BRAM/DSP не опубликованы; target xczu15eg	DVP clock 100 МГц в demo script; core frequency не опубликована
NPU_X_ Interface	Convolution operations, Im2Col, systolic array, psum management, RISC-V X-Interface	Оценка RTL: 3.28 Mbit internal SRAM ≈ 90–110 RAMB36; до 288 DSP теоретически; LUT unknown	Не опубликована; инженерная оценка 100–200 МГц
FINN	QNN/BNN, FC, Conv, Pooling, network-specific streaming dataflow	Примеры: 5155–91131 LUT; 4.5–396 BRAM; DSP не ключевая метрика для BNN	200 МГц в FINN paper

Решение	**Поддерживаемые модели/слои**	**Ресурсы LUT/BRAM/DSP**	**Частота**
hls4ml	MLP, CNN, RNN/LSTM/GRU; Keras/PyTorch/ONNX; fixed-point/quantized	Пример CNN QP 7-bit: 48259 LUT, 122 BRAM18K, 213 DSP на XC7Z020	100 МГц в опубликованном примере

Таблица 4 – Сравнение рассмотренных решений: производительность и статус данных

Решение	**Latency / throughput**	**Статус данных**
Tiny TPU	End-to-end latency не опубликована; требуется измерение cycles от start до done.	Quartus reports опубликованы для нескольких demo-конфигураций.
FREE-TPU V3+	ASIC 1 ГГц reference: MobileNetV2 INT8 1.5 ms, ResNet50 INT8 6.19 ms, YOLOv5s INT8 14.19 ms.	FPGA utilization отсутствует; latency относится к ASIC reference, а не к FPGA.
NPU_X_ Interface	Теоретический peak: 28.8–57.6 GMAC/s при 100–200 МГц; latency не опубликована.	Только предварительная RTL-оценка; точные Vivado reports отсутствуют.
FINN	MNIST SFC-max: 12.361M FPS, 0.31 μs; CNV-max: 21.9k FPS, 283 μs.	Опубликованная статья; ресурсы зависят от модели и folding.
hls4ml	171 μs; 2831 frames/s для QP 7-bit CNN.	Опубликованная статья; характеристики зависят от модели, precision и reuse factor.

Из таблицы видно, что решения относятся к разным классам. Tiny TPU полезен как открытый компактный RTL/reference с опубликованными FPGA-ресурсами. FREE-TPU V3+ ближе к промышленному edge-inference IP, но менее удобен для академического RTL-анализа из-за encrypted IP и отсутствия открытых FPGA utilization reports. NPU_X_Interface хорошо раскрывает архитектуру convolution pipeline, но требует локального синтеза для получения точных ресурсов. FINN и hls4ml являются не отдельными фиксированными accelerator cores, а toolflow/framework-решениями, поэтому их результаты зависят от модели, precision и параметров генерации.

1.4 Преимущества, ограничения и требования к плате

Таблица 5 – Преимущества и ограничения рассмотренных решений

Решение	**Преимущества**	**Ограничения**
Tiny TPU	Открытый RTL, понятная архитектура, опубликованные Quartus reports, низкие требования к FPGA, удобен для анализа systolic array.	Ограниченная поддержка моделей, нет универсального compiler flow, отсутствуют опубликованные end-to-end latency/throughput для всех demo.
FREE-TPU V3+	Реальные CNN/RNN/LSTM workloads, SDK/compiler, INT8/FP16, современная edge-inference направленность.	Encrypted IP, FPGA utilization не опубликован, one-hour limitation, требуется крупная ZynqMP FPGA, слабая пригодность для RTL-анализа.
NPU_X_ Interface	Открытый SystemVerilog, хорошая декомпозиция convolution pipeline, интеграция с RISC-V, полезен для архитектурного анализа.	Нет Vivado reports, нет benchmark, нет compiler/model deployment flow, ресурсы можно оценивать только предварительно.
FINN	Очень высокая throughput/low latency, зрелый quantized dataflow approach, хорошая академическая база, генерация accelerator под сеть.	Требует quantization и Xilinx flow, сложнее для быстрого воспроизведения, результат зависит от модели/folding.

Решение	**Преимущества**	**Ограничения**
hls4ml	Практичный HLS path от ML-модели к FPGA, поддержка Keras/PyTorch/ONNX, гибкость precision/reuse factor, rapid prototyping.	Не фиксированный accelerator, ресурсы могут быть высокими, для каждого случая нужен synthesis, большие CNN требуют крупных FPGA.

Таблица 6 – Оценка требований к отладочной плате

Решение	**Подходящая плата/класс FPGA**	**Обоснование**
Tiny TPU	DE1-SoC / Cyclone V	Максимальная опубликованная MNIST JTAG-конфигурация использует 22% ALM, 25% RAM blocks и 8% DSP.
FREE-TPU V3+	Zynq UltraScale+ MPSoC уровня ZU15EG	Demo script ориентирован на xczu15eg; точные ресурсы необходимо получить локально.
NPU_X_ Interface	Крупная Xilinx FPGA, например VCU118/XCVU9P	Внутренняя SRAM-оценка около 3.28 Mbit и возможное использование сотен DSP; нужна плата с большим запасом.
FINN	ZC706, ZCU104, PYNQ/Ultra96/Alveo в зависимости от модели	FINN генерирует accelerator под сеть; требования зависят от folding и размера модели.
hls4ml	От PYNQ-Z2/XC7Z020 для малых CNN до более крупных Xilinx FPGA	Опубликованный CNN QP 7-bit помещается в XC7Z020, но использует почти весь ресурс LUT/DSP.

1.5 Выбор направления дальнейшей реализации

По результатам сравнения открытых решений было принято решение не воспроизводить полностью одну из крупных accelerator-платформ, таких как FINN, VTA или FREE-TPU V3+, а выбрать более компактное и воспроизводимое направление: реализацию небольшого CNN inference accelerator с использованием HLS/RTL-потока и последующей проверкой на отладочной плате FPGA.

Такой выбор был сделан по нескольким причинам. Во-первых, рассмотренные крупные фреймворки обладают более сложной инструментальной цепочкой и часто требуют специальной подготовки модели, настройки toolflow или использования конкретной аппаратной платформы. Это затрудняет их прямое воспроизведение в рамках ограниченного этапа практики. Во-вторых, для выполнения индивидуального задания было важно получить не только теоретическое сравнение, но и собственный проверяемый результат: C-симуляцию, HLS-синтез, RTL-интеграцию, implementation в Vivado и аппаратный запуск на FPGA. В-третьих, компактная HLS-реализация позволяет сохранить архитектурную прозрачность и затем локально исследовать влияние отдельных доработок, например изменения организации сверточного ядра.

В качестве реализуемого направления был выбран компактный inference-ориентированный CNN-ускоритель, включающий основные операции сверточной сети: convolution, ReLU, max pooling, flatten и fully connected layer. Такой вариант не является прямой копией одного из рассмотренных проектов, однако использует идеи, выделенные в ходе обзора: модульную организацию вычислительного тракта, low-precision представление данных, локальную буферизацию и возможность последующего улучшения convolution core.

Отдельно был выполнен выбор целевой аппаратной платформы. Для практической проверки была использована отладочная плата Nexys4 DDR на базе FPGA Artix-7 xc7a100tcsg324-3. Эта плата была выбрана потому, что она доступна для экспериментов, поддерживается используемой версией Vivado/Vivado HLS 2019.1 и имеет достаточный запас ресурсов для компактного CNN-ускорителя. Согласно отчету Vivado для выбранной FPGA доступны 63400 LUT, 126800 триггеров, 135 блоков BRAM и 240 DSP-блоков. Эти ресурсы существенно превышают предварительные требования проектируемого ускорителя, что позволяет не только реализовать контрольную конфигурацию, но и выполнить экспериментальную архитектурную доработку без риска исчерпания ресурсов платы.

Выбор частоты 100 МГц связан с типовым ограничением для учебного FPGA-проекта на Artix-7 и позволяет упростить анализ временных характеристик. При этом такая частота достаточна для оценки latency, проверки timing closure и сравнения контрольной и оптимизированной HLS-конфигураций. Использование memory-mapped управления через UDM также было важно для практической части, поскольку позволило проверить не только вычислительное ядро, но и полный путь взаимодействия: запись входного изображения, запуск ускорителя, ожидание признака завершения и чтение выходных оценок классов.

Таким образом, выбранное направление представляет собой компромисс между современными архитектурными идеями, выявленными при анализе открытых решений, и практической воспроизводимостью в рамках доступной аппаратной и программной среды. Оно позволяет выполнить полный цикл разработки и проверки CNN-ускорителя на FPGA, а также получить количественные результаты по ресурсам, частоте, latency, timing и оценке энергопотребления.

2 Запуск выбранного проекта на отладочной плате ПЛИС и сборка

2.1 Подготовка контрольной CNN-конфигурации

2.1.1 Структура вычислительного графа

Для практического запуска была выбрана компактная сверточная нейронная сеть, достаточная для демонстрации полного inference-потока на FPGA. Архитектура не претендует на промышленную точность на полном наборе данных, но позволяет проверить основные операции CNN: свертку, нелинейность, pooling, flatten и полносвязный слой.

Таблица 7 – Архитектура реализованной CNN

Этап	**Параметры**	**Выходной размер**
Вход	Одноканальное изображение	28×28
Convolution	4 фильтра размером 5×5, stride = 1, padding = 0	24×24×4
Activation	ReLU с ограничением диапазона int8	24×24×4
MaxPool	Окно 2×2, шаг 2	12×12×4
Flatten	Развертывание feature maps	576

Этап	**Параметры**	**Выходной размер**
Fully connected	Полносвязный слой на 10 классов	10 оценок классов

Размерность flatten-слоя определяется выражением $12 \times 12 \times 4 = 576$. Выходной слой формирует десять целочисленных оценок, после чего класс определяется по максимальному значению.

2.1.2 Типы данных и программная структура

Таблица 8 – Типы данных в HLS-реализации

Тип	**Назначение**	**Разрядность**
data_t	входные пиксели и промежуточные данные	int8_t
weight_t	веса и смещения модели	int8_t
acc_t	аккумуляторы и выходные оценки классов	int32_t

Использование 8-разрядных данных и весов согласуется с выбранным направлением low-precision inference. Для накопления произведений используется 32-разрядный тип, что уменьшает риск переполнения при суммировании большого числа слагаемых.

Таблица 9 – Структура программной HLS-реализации

Функция	**Назначение**
conv_layer	вычисление четырех карт признаков с ядрами 5×5 и последующим ReLU/clipping
maxpool_layer	понижение пространственного размера с использованием max pooling 2×2
flatten_layer	преобразование трехмерного массива feature maps в одномерный вектор
fc_layer	полносвязная классификация по десяти классам
cnn_accel	верхняя HLS-функция, объединяющая все этапы вычислений

Верхняя функция вызывает слои последовательно: convolution, maxpool, flatten и fully connected. Такая организация была использована как контрольная

конфигурация: она проста для проверки, дает рабочую контрольную конфигурацию и позволяет затем оценить эффект архитектурных доработок.

2.2 Процесс проверки контрольной реализации

2.2.1 C-симуляция HLS-модели

На этапе C-симуляции использовался тестовый C/C++ testbench. Он формирует синтетические изображения цифр размером 28×28, передает их в функцию `cnn_accel` и выводит десять оценок классов. Проверка подтвердила корректность вычислительного тракта на уровне алгоритмической модели.

Таблица 10 – Результаты функциональной проверки контрольной HLS-конфигурации

Тестовое изображение	**Предсказанный класс**	**Лучшая оценка**
digit 1	1	1784
digit 7	7	1925
digit 0	0	506

2.2.2 HLS-синтез контрольной конфигурации

После C-симуляции был выполнен HLS-синтез для семейства Artix-7. В качестве целевого периода тактирования был задан период 10 нс, соответствующий частоте 100 МГц. HLS-оценка показала, что расчетная длительность критического пути составляет 7.777 нс, то есть укладывается в заданное ограничение.

Таблица 11 – Основные результаты HLS-синтеза контрольной конфигурации

Параметр	**Значение**
Target clock	10.00 нс
Estimated clock	7.777 нс
Latency	226449 тактов
Оценка времени при 100 МГц	2.26 мс
BRAM_18K	6
DSP48E	2
FF	932
LUT	1840

Низкое использование ресурсов показывает, что выбранная CNN является

компактной и может служить контрольной конфигурацией для последующей инженерной оценки. Одновременно высокая latency в тактах отражает последовательный характер исходной версии без агрессивной конвейеризации и распараллеливания.

2.2.3 RTL-интеграция и memory-mapped интерфейс

После HLS-синтеза RTL-описание модуля `cnn_accel` было интегрировано в проект верхнего уровня `NEXYS4_DDR`. В составе проекта присутствуют модуль CNN-ускорителя, модуль тактирования, синхронизация сброса, тестовая память и UDM-интерфейс для отладочного доступа к памяти и регистрам.

Для взаимодействия с ускорителем использована memory-mapped схема. Входное изображение записывается в область памяти, затем через управляющий регистр формируется запуск ускорителя. После завершения вычислений признак готовности считывается из статусного регистра, а десять выходных оценок классов читаются из области результатов.

Таблица 12 – Карта адресов memory-mapped интерфейса CNN

Адрес	**Назначение**	**Размер/смысл**
0x10000000	CNN_CTRL_ADDR	регистр запуска ускорителя
0x10000004	CNN_STATUS_ADDR	статус завершения и готовности
0x10001000	CNN_IMAGE_BASE	входное изображение 28×28, 784 слова
0x10002000	CNN_RESULT_BASE	10 выходных оценок классов

Сохранение этой карты адресов стало важным для дальнейших экспериментов: при замене внутренней реализации CNN-модуля внешняя схема управления и Python-скрипт могли оставаться неизменными.

2.2.4 RTL-симуляция интегрированного проекта

RTL-симуляция выполнялась уже не как прямой вызов C-функции, а как проверка интегрированного проекта через UDM-транзакции. Testbench записывает входные пиксели по адресам `CNN_IMAGE_BASE`, запускает вычислитель, ожидает завершения, после чего считывает десять 32-разрядных выходных значений из области `CNN_RESULT_BASE`.

Таблица 13 – Результаты RTL-симуляции интегрированного проекта

Тестовое изображение	**Предсказанный класс**	**Лучшая оценка**	**Время завершения симуляции**
digit 0	0	506	2.501 мс
digit 7	7	1925	2.501 мс

Полученные результаты совпали с ожидаемыми классами и с результатами C-симуляции. Это подтверждает корректность переноса вычислительного блока в RTL-систему и корректность memory-mapped управления.

2.2.5 Синтез, implementation, timing и power analysis

После интеграции был выполнен синтез полного проекта `NEXYS4_DDR`. В отличие от HLS-оценки, приведенные ниже значения относятся к полной RTL-системе, включающей CNN-ускоритель, UDM-интерфейс, тестовую память и верхний модуль платы.

Таблица 14 – Использование ресурсов после синтеза полного интегрированного проекта

Ресурс	**Использовано**	**Доступно**	**Использование, %**
Slice LUTs	1167	63400	1.84
LUT as Logic	959	63400	1.51
LUT as Memory	208	19000	1.09
Slice Registers	999	126800	0.79
Block RAM Tile	4	135	2.96
DSPs	2	240	0.83
Bonded IOB	35	210	16.67

Таблица 15 – Временные характеристики базовой версии после implementation

Параметр	**Значение**
Worst Negative Slack (WNS)	3.696 нс
Total Negative Slack (TNS)	0.000 нс
Number of Failing Endpoints	0
Worst Hold Slack (WHS)	0.055 нс

Параметр	**Значение**
Total Hold Slack (THS)	0.000 нс
Worst Pulse Width Slack (WPWS)	3.000 нс

Таблица 16 – Оценка энергопотребления базовой версии после routing

Параметр	**Значение**
Total On-Chip Power	0.221 Вт
Dynamic Power	0.124 Вт
Device Static Power	0.098 Вт
Junction Temperature	26.0 °C
cnn_accel_inst	0.011 Вт
sys_clk/MMCM	0.106 Вт

Положительное значение WNS и нулевое значение TNS показывают, что заданные временные ограничения выполнены. Оценка мощности является расчетной оценкой Vivado после routing, а не прямым измерением внешним прибором.

2.2.6 Аппаратная проверка на FPGA

После формирования и загрузки проекта на плату была выполнена аппаратная проверка. Python-скрипт подключался к COM-порту, записывал изображение во входную память, запускал CNN, ожидал признак завершения и считывал выходные оценки классов.

Таблица 17 – Результаты аппаратного тестирования базовой версии на FPGA

Тестовое изображение	**Статус**	**Предсказанный класс**	**Лучшая оценка**
digit 7	0x3	7	1925
digit 1	0x3	1	1784

Статус 0x3 соответствует успешному завершению вычисления и готовности блока к дальнейшей работе. Результаты аппаратного запуска совпали с результатами предыдущих уровней проверки.

2.3 Архитектурная доработка сверточного ядра

2.3.1 Мотивация доработки

После проверки контрольной версии была разработана экспериментальная оптимизированная версия сверточного ядра. Цель доработки состояла не в изменении внешнего интерфейса ускорителя, а во внутреннем улучшении организации вычислений convolution layer. Это позволило сохранить уже реализованную систему управления, UDM-интерфейс и карту адресов, но изменить структуру вычислительного тракта внутри слоя свертки.

В базовой версии свертка выполнялась прямым способом: для каждого выходного пикселя, каждого фильтра и каждой позиции ядра выполнялось обращение к массиву входного изображения. Такая схема удобна для первичной проверки, но с аппаратной точки зрения не является оптимальной, поскольку одни и те же входные пиксели многократно используются при вычислении соседних выходных значений.

2.3.2 Структура line buffer и MAC-ядра

В оптимизированной версии введен локальный line buffer размером 5×28, соответствующий пяти строкам входного изображения при размере ядра 5×5. Каждая новая строка последовательно загружается в буфер. После накопления первых пяти строк можно формировать локальное окно свертки и вычислять выходную строку.

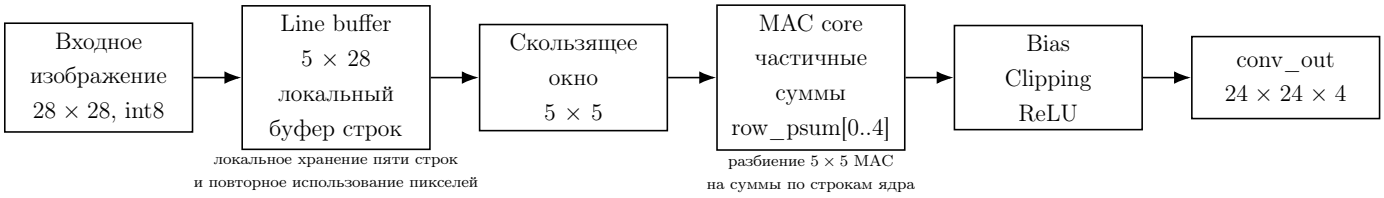


Рисунок 1 – Упрощенная структура оптимизированного сверточного ядра

Вычисление MAC-операции было выделено в отдельную функцию. Внутри нее формируются частичные суммы по строкам ядра: `row_psum[0]`, ..., `row_psum[4]`. Затем они суммируются вместе с bias соответствующего фильтра. Для повышения степени параллелизма применены HLS-директивы `INLINE`, `ARRAY_PARTITION`, `UNROLL` и `PIPELINE`.

Таблица 18 – Ключевые изменения в оптимизированной версии

Параметр	**Изменение**
Доступ к входным данным	Вместо многократного обращения к исходному массиву используется локальный line buffer.
Организация окна	Пять строк изображения хранятся локально и переиспользуются для соседних окон.
MAC-операция	Выделена отдельная функция `conv5x5_linebuf_mac`.
Частичные суммы	Для каждой строки ядра формируется отдельная row partial sum.
Внешний интерфейс	Не изменен; memory-mapped интеграция остается прежней.

2.3.3 Проверка оптимизированной версии

Функциональная проверка оптимизированной версии на уровне C-симуляции показала корректную классификацию тестового изображения digit 1: предсказанный класс 1, best score 1784. После этого был выполнен HLS-синтез, который показал снижение максимальной латентности с 226449 до 199933 тактов.

Таблица 19 – Сравнение HLS-результатов базовой и оптимизированной версий

Параметр	**Базовая версия**	**Оптимизированная версия**
Максимальная latency, такты	226449	199933
Оценка времени при 100 МГц	2.26 мс	2.00 мс
BRAM_18K	6	6
DSP48E	2	11
FF	932	1765
LUT	1840	3320
Estimated clock	7.777 нс	7.777 нс

После integration и implementation оптимизированной версии были получены следующие результаты полного проекта.

Таблица 20 – Использование ресурсов оптимизированной версии после implementation

Ресурс	**Полный проект NEXYS4_DDR**	**Экземпляр cnn_accel_inst**
Slice LUTs	1668	933
Slice Registers	1316	724
F7 Muxes	101	4
Block RAM Tile	4	3
DSPs	11	11
Bonded IOB	36	0
BUFGCTRL	2	0
MMCME2_ADV	1	0

Таблица 21 – Timing оптимизированной версии после implementation

Параметр	**Значение**
Worst Negative Slack (WNS)	3.811 нс
Total Negative Slack (TNS)	0.000 нс
Number of Failing Endpoints	0
Worst Hold Slack (WHS)	0.070 нс
Total Hold Slack (THS)	0.000 нс
Worst Pulse Width Slack (WPWS)	3.000 нс

Таблица 22 – Оценка мощности оптимизированной версии

Параметр	**Значение**
Total On-Chip Power	0.231 Вт
Dynamic Power	0.133 Вт
Device Static Power	0.098 Вт
Junction Temperature	26.1 °C
Thermal Margin	73.9 °C

Оптимизированная версия также была проверена на реальной FPGA. Для тестового изображения digit 7 был получен корректный результат: предсказанный класс 7, best score 1829.

2.4 Сопоставление полученных результатов

2.4.1 Сравнение контрольной и оптимизированной реализаций

Таблица 23 – Итоговое сравнение базовой и оптимизированной версий

Параметр	**Базовая версия**	**Оптимизированная версия**
HLS max latency, такты	226449	199933
Оценка времени при 100 МГц	2.26 мс	2.00 мс
Slice LUTs, полный проект	1167	1668
Slice Registers, полный проект	999	1316
Block RAM Tile, полный проект	4	4
DSPs, полный проект	2	11
WNS после implementation	3.696 нс	3.811 нс
TNS после implementation	0.000 нс	0.000 нс
Total On-Chip Power	0.221 Вт	0.231 Вт
FPGA-проверка digit 7	class 7	class 7

По результатам сравнения контрольной и оптимизированной реализаций я сделал вывод, что уменьшение латентности в FPGA-проектировании не является бесплатным. Оптимизированная версия позволила снизить максимальную HLS-латентность примерно на 11.7%, однако для этого потребовалось увеличить степень аппаратного параллелизма, что привело к росту использования вычислительных ресурсов, прежде всего DSP-блоков. На этом этапе я на практике увидел типичный для FPGA-проектирования компромисс между производительностью и аппаратными затратами: ускорение вычислений достигается за счет большего числа параллельно работающих аппаратных блоков. При этом обе версии выполнили временные ограничения на целевой частоте и были проверены на FPGA, что подтвердило корректность выбранного маршрута разработки и сравнения.

2.5 Итоги выполненных работ в ходе практики

В ходе практики я прошел полный путь от изучения существующих открытых решений до получения работающего прототипа CNN-ускорителя на FPGA. Для меня важным результатом стало не только получение корректных численных выходов модели, но и понимание того, как отдельные этапы FPGA-разработки связаны между собой: архитектурный анализ, выбор реализуемого направления, HLS-описание, синтез, RTL-интеграция, implementation и аппаратная проверка.

Сначала я изучил несколько открытых проектов и инструментальных экосистем, связанных с реализацией нейросетевых вычислений на FPGA. Этот этап помог мне понять, что прямое воспроизведение крупной accelerator-платформы не всегда является рациональным путем для учебно-исследовательской практики. Более подходящим оказался инкрементальный подход: взять из рассмотренных проектов ключевые архитектурные идеи и реализовать компактный, понятный и проверяемый CNN inference accelerator.

После выбора направления я подготовил HLS-совместимую реализацию небольшой сверточной сети. На этом этапе я лучше понял, как алгоритмическая структура CNN преобразуется в аппаратно-ориентированное описание: входное изображение, сверточный слой, ReLU, max pooling, flatten и полносвязный слой должны быть представлены не только как математическая модель, но и как последовательность вычислительных блоков, пригодных для синтеза.

Следующим важным этапом стала проверка реализации на уровне C-симуляции и HLS-синтеза. Здесь я получил первый практический опыт оценки latency, estimated clock и использования ресурсов LUT, BRAM, DSP и FF. Это позволило увидеть разницу между функционально корректной программной моделью и аппаратной реализацией, для которой важны не только правильные выходные значения, но и временные характеристики, структура интерфейса и использование ресурсов FPGA.

После HLS-синтеза я интегрировал полученный RTL-модуль в проект верхнего уровня для платы Nexys4 DDR. Этот этап был особенно полезен, поскольку показал, что наличие синтезируемого HLS-модуля еще не означает готовность проекта к запуску на FPGA. Необходимо также обеспечить корректное управление модулем, согласовать memory-mapped адреса, проверить запуск, статус завершения и чтение выходных данных через тестовую инфраструктуру.

RTL-симуляция и аппаратная проверка через Python/UDM помогли мне

убедиться, что вычислительный модуль работает не только как отдельная HLS-функция, но и как часть интегрированной FPGA-системы. Практически важным результатом стало совпадение предсказанных классов на разных уровнях проверки: C-симуляция, RTL-симуляция и запуск на реальной плате.

Отдельным результатом практики стала экспериментальная доработка сверточного ядра с использованием line buffer и частичных сумм. При выполнении этой доработки я увидел, что оптимизация FPGA-проекта требует анализа компромисса между производительностью и ресурсами. Уменьшение максимальной HLS-латентности было достигнуто за счет увеличения аппаратного параллелизма и, соответственно, роста использования DSP, LUT и FF. Этот результат помог мне лучше понять, что архитектурные улучшения в FPGA-проектах необходимо оценивать не только по latency, но и по полной совокупности характеристик: ресурсы, timing, мощность и возможность аппаратной проверки.

Таким образом, в ходе практики я получил опыт анализа открытых решений, выбора реализуемого направления, разработки HLS-модели, интеграции RTL-модуля, работы с отчетами Vivado, проверки timing closure, оценки мощности и аппаратного тестирования на FPGA. Полученный прототип можно рассматривать как основу для дальнейшего исследования архитектурных вариантов CNN-ускорителей на ПЛИС.

Таблица 24 – Основные результаты практики

Направление	**Полученный результат**
Обзор решений	Выполнено сравнение Tiny TPU, FREE-TPU V3+, NPU_X_Interface, FINN и hls4ml по ключевым инженерным характеристикам.
Выбор направления	Обоснован выбор компактной HLS/RTL-реализации CNN inference accelerator с заимствованием идей открытых проектов.
Контрольная реализация	Получена рабочая HLS/RTL-реализация CNN с latency 226449 тактов и успешным запуском на FPGA.
Vivado implementation	Полный проект базовой версии использует 1167 Slice LUTs, 999 Slice Registers, 4 BRAM Tile и 2 DSP; WNS = 3.696 нс.

Направление	**Полученный результат**
Аппаратная проверка	На FPGA получены корректные результаты для digit 7 и digit 1.
Оптимизация	Версия с line buffer уменьшила HLS max latency до 199933 тактов и прошла FPGA-проверку для digit 7.

2.6 Направления дальнейшей работы

Полученная реализация является рабочей базой для дальнейших экспериментов. Дальнейшие шаги целесообразно определять с учетом рекомендаций научного руководителя. В общем виде развитие работы может включать углубленный анализ вариантов HLS-оптимизации, улучшение организации обмена данными, более точное измерение чистой latency вычислительного ядра и расширение набора тестовых изображений.

Отдельный интерес представляет дальнейшее исследование компромисса между latency и использованием ресурсов. Оптимизированная версия показала, что снижение latency сопровождается увеличением использования DSP и LUT. Поэтому на следующих этапах может быть полезно сравнить несколько вариантов разворачивания циклов, локальной буферизации, конвейеризации и возможного перехода к более выраженной streaming/dataflow-архитектуре.

ЗАКЛЮЧЕНИЕ

В ходе производственной практики была выполнена работа по теме аппаратной реализации сверточных нейронных сетей на ПЛИС. Работа включала не только разработку собственного прототипа, но и предварительный анализ открытых решений, необходимых для выбора реалистичного направления дальнейшей реализации.

На первом этапе я проанализировал открытые проекты и инструментальные экосистемы, связанные с FPGA-ускорителями нейронных сетей: Tiny TPU, FREE-TPU V3+ for FPGA, NPU_X_Interface, FINN и hls4ml. Этот анализ помог мне понять, что прямое воспроизведение крупной accelerator-платформы не всегда является оптимальным путем для учебно-исследовательской практики. В результате был выбран инкрементальный подход: реализация компактного CNN inference accelerator с использованием HLS/RTL-маршрута и последующей проверкой на доступной отладочной плате FPGA.

Практическая часть работы позволила пройти полный инженерный цикл разработки: подготовку HLS-описания, C-симуляцию, HLS-синтез, интеграцию RTL-модуля в проект верхнего уровня, RTL-симуляцию, synthesis и implementation в Vivado, анализ временных характеристик, оценку мощности и аппаратный запуск на FPGA. На этом этапе я убедился, что корректная HLS-модель сама по себе еще не является завершенным FPGA-проектом: необходимо также проверить интерфейс управления, memory-mapped взаимодействие, timing closure и работоспособность проекта на реальной плате.

Контрольная реализация CNN-ускорителя была успешно проверена на нескольких уровнях. Результаты C-симуляции, RTL-симуляции и аппаратного запуска совпали по предсказанным классам для тестовых изображений. Это подтвердило корректность переноса вычислительной модели из HLS-описания в RTL-проект и далее на физическую FPGA. По результатам HLS-синтеза и Vivado implementation были получены количественные характеристики по latency, LUT, BRAM, DSP, FF, timing и оценке энергопотребления.

Дополнительно была разработана и проверена оптимизированная версия сверточного ядра с использованием line buffer и частичных сумм. При сравнении контрольной и оптимизированной реализаций я на практике увидел характерный для FPGA-проектирования компромисс: уменьшение latency достигается за счет большего аппаратного параллелизма и, соответственно, увеличения использования

вычислительных ресурсов. Оптимизированная версия уменьшила максимальную HLS-латентность примерно на 11.7%, однако потребовала большего числа DSP-блоков, LUT и регистров. При этом обе версии выполнили временные ограничения на целевой частоте и были проверены на FPGA.

Таким образом, цель практики была достигнута. Были выполнены обзор и сравнение открытых решений, обоснован выбор направления дальнейшей реализации, создан и проверен компактный CNN-ускоритель на FPGA, получены экспериментальные результаты на уровнях HLS, RTL, Vivado implementation и аппаратного запуска, а также выполнена первая архитектурная доработка сверточного ядра. Полученный прототип может использоваться как основа для дальнейшего исследования архитектурных вариантов CNN-ускорителей на ПЛИС, включая более глубокую оптимизацию dataflow, локальной буферизации и степени аппаратного параллелизма.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] tiny-tpu-v2, “tiny-tpu,”[Электронный ресурс]. — Режим доступа: https://github.com/tiny-tpu-v2/tiny-tpu

[2] embedeep, “FREE-TPU-V3plus-for-FPGA,” [Электронный ресурс]. — Режим доступа: https://github.com/embedeep/FREE-TPU-V3plus-for-FPGA

[3] H. Ouinti, “NPU_X_Interface,” [Электронный ресурс]. — Режим доступа: https://github.com/habibaouinti/NPU_X_Interface

[4] Xilinx, “FINN,” [Электронный ресурс]. — Режим доступа: https://github.com/Xilinx/finn

[5] Fast Machine Learning Lab, “hls4ml,” [Электронный ресурс]. — Режим доступа: https://github.com/fastmachinelearning/hls4ml

[6] Apache TVM, “VTA,” [Электронный ресурс]. — Режим доступа: https://github.com/apache/tvm-vta

[7] D. Wang et al., “PipeCNN,” [Электронный ресурс]. — Режим доступа: https://github.com/doonny/PipeCNN

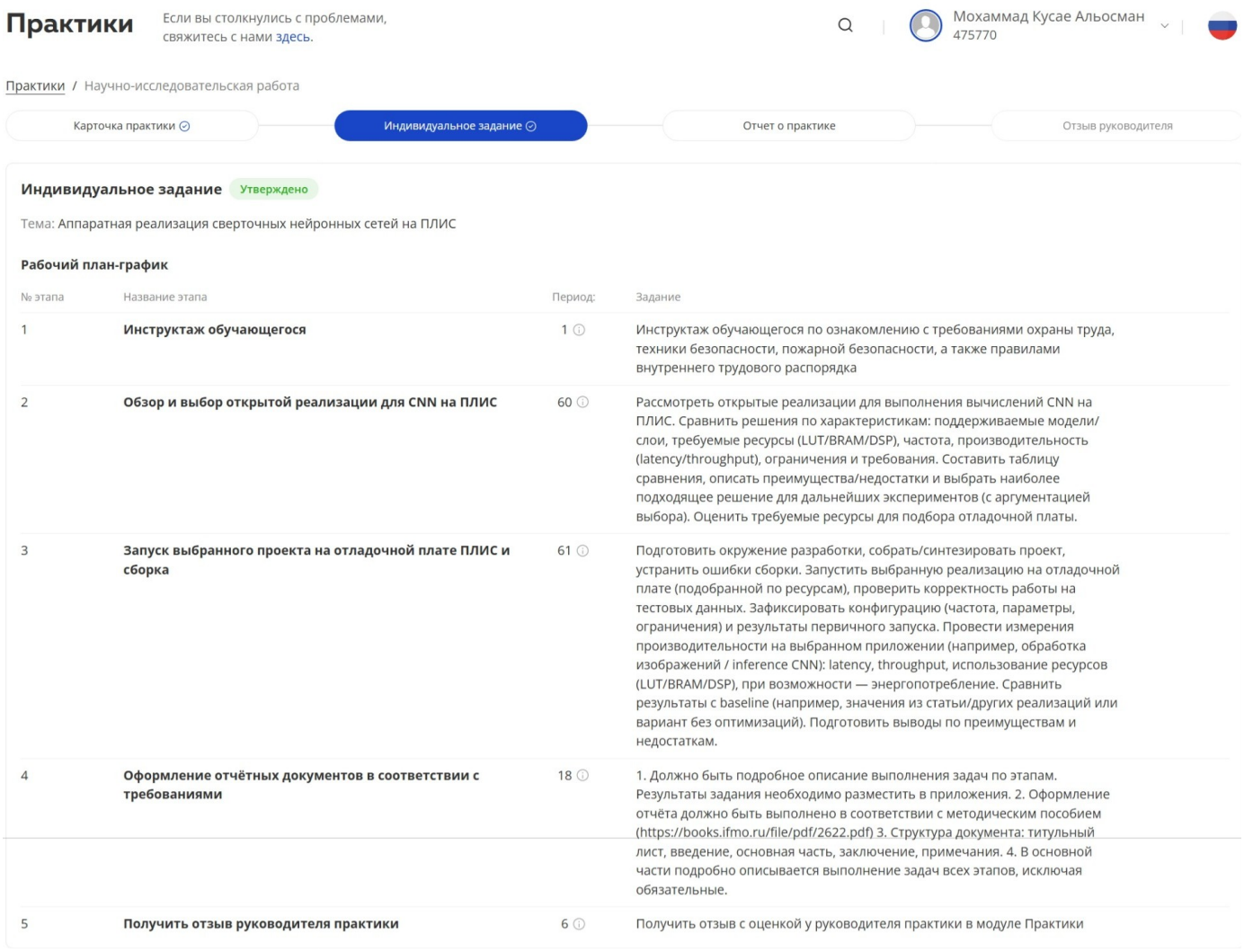
[8] A. Montgomerie, “fpgaConvNet HLS,” [Электронный ресурс]. — Режим доступа: https://github.com/AlexMontgomerie/fpgaconvnet-hls

[9] Xilinx, “Vivado Design Suite User Guide: High-Level Synthesis (UG902),” [Электронный ресурс]. — Режим доступа: https://docs.xilinx.com/r/en-US/ug902-vivado-high-level-synthesis

[10] Digilent, “Nexys4 DDR Reference Manual,” [Электронный ресурс]. — Режим доступа: https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual

ПРИЛОЖЕНИЕ А

План-график выполнения индивидуального задания



Практики Если вы столкнулись с проблемами, свяжитесь с нами здесь.

Мохаммад Кусае Альосман
475770

Практики / Научно-исследовательская работа

Карточка практики | Индивидуальное задание | Отчет о практике | Отзыв руководителя

Индивидуальное задание Утверждено

Тема: Аппаратная реализация сверточных нейронных сетей на ПЛИС

Рабочий план-график

№ этапа	Название этапа	Период:	Задание
1	**Инструктаж обучающегося**	1	Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка
2	**Обзор и выбор открытой реализации для CNN на ПЛИС**	60	Рассмотреть открытые реализации для выполнения вычислений CNN на ПЛИС. Сравнить решения по характеристикам: поддерживаемые модели/слои, требуемые ресурсы (LUT/BRAM/DSP), частота, производительность (latency/throughput), ограничения и требования. Составить таблицу сравнения, описать преимущества/недостатки и выбрать наиболее подходящее решение для дальнейших экспериментов (с аргументацией выбора). Оценить требуемые ресурсы для подбора отладочной платы.
3	**Запуск выбранного проекта на отладочной плате ПЛИС и сборка**	61	Подготовить окружение разработки, собрать/синтезировать проект, устранить ошибки сборки. Запустить выбранную реализацию на отладочной плате (подобранной по ресурсам), проверить корректность работы на тестовых данных. Зафиксировать конфигурацию (частота, параметры, ограничения) и результаты первичного запуска. Провести измерения производительности на выбранном приложении (например, обработка изображений / inference CNN): latency, throughput, использование ресурсов (LUT/BRAM/DSP), при возможности — энергопотребление. Сравнить результаты с baseline (например, значения из статьи/других реализаций или вариант без оптимизаций). Подготовить выводы по преимуществам и недостаткам.
4	**Оформление отчётных документов в соответствии с требованиями**	18	1. Должно быть подробное описание выполнения задач по этапам. Результаты задания необходимо разместить в приложения. 2. Оформление отчёта должно быть выполнено в соответствии с методическим пособием (https://books.ifmo.ru/file/pdf/2622.pdf) 3. Структура документа: титульный лист, введение, основная часть, заключение, примечания. 4. В основной части подробно описывается выполнение задач всех этапов, исключая обязательные.
5	**Получить отзыв руководителя практики**	6	Получить отзыв с оценкой у руководителя практики в модуле Практики

Рисунок 2 – План-график выполнения работ в рамках практики

ПРИЛОЖЕНИЕ Б

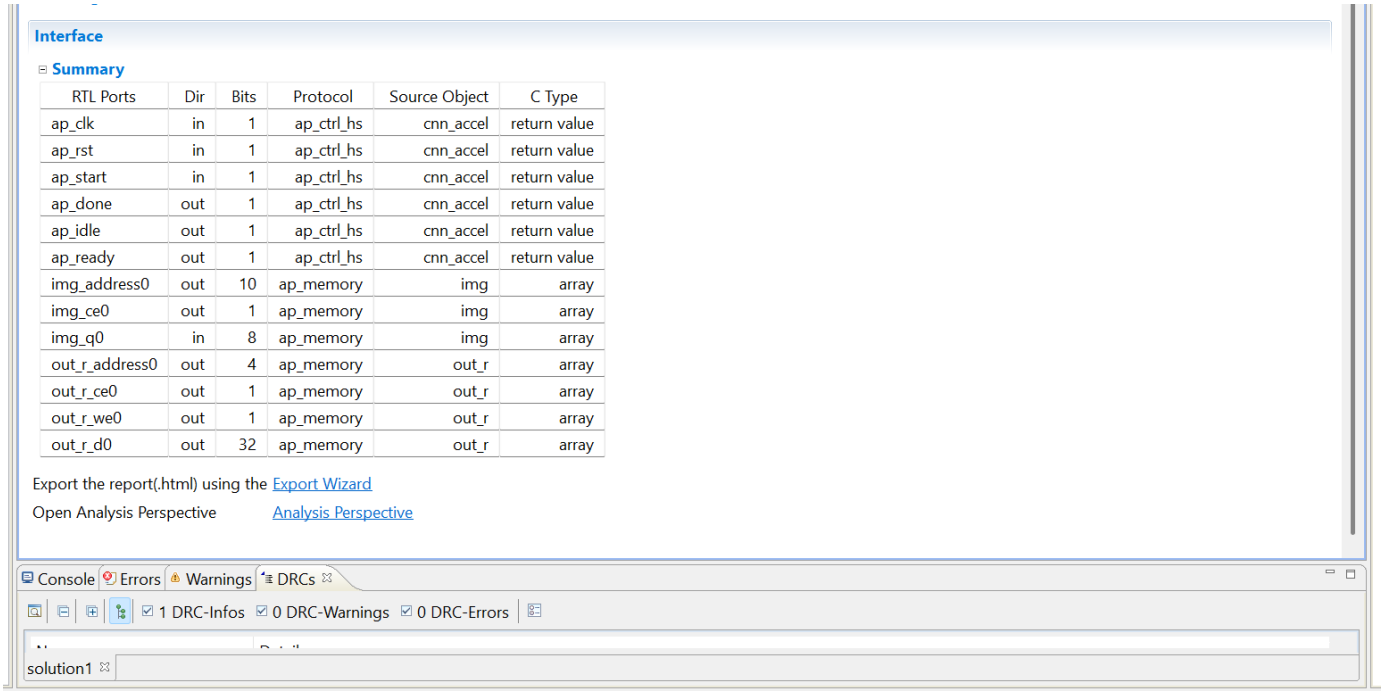
Подробные сведения по сравнению открытых решений

Таблица 25 – Расширенные комментарии к рассмотренным открытым решениям

Решение	**Комментарий**
Tiny TPU	Наиболее полезен как компактный открытый RTL/reference. Наличие опубликованных Quartus-отчетов делает его удобным для оценки ресурсов, однако отсутствует полноценный compiler flow для произвольных CNN.
FREE-TPU V3+	Показывает промышленно-ориентированный путь edge inference с SDK/compiler и поддержкой реальных моделей, но закрытость IP и отсутствие открытых FPGA utilization reports ограничивают его применимость для академического ресурсного анализа.
NPU_X_Interface	Ценен как архитектурная декомпозиция convolution accelerator: input staging, weights staging, systolic array, psum management. Точные ресурсы требуют локального Vivado-синтеза.
FINN	Используется как ориентир зрелой quantized dataflow-архитектуры. Результаты впечатляющие, но зависят от модели, folding и precision; toolflow сложнее для быстрого воспроизведения.
hls4ml	Подтверждает практическую состоятельность HLS-based маршрута от ML-модели к FPGA. Ресурсы и latency сильно зависят от precision и reuse factor.

ПРИЛОЖЕНИЕ В

Материалы HLS и RTL-проверки контрольной версии



Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	cnn_accel	return value
ap_rst	in	1	ap_ctrl_hs	cnn_accel	return value
ap_start	in	1	ap_ctrl_hs	cnn_accel	return value
ap_done	out	1	ap_ctrl_hs	cnn_accel	return value
ap_idle	out	1	ap_ctrl_hs	cnn_accel	return value
ap_ready	out	1	ap_ctrl_hs	cnn_accel	return value
img_address0	out	10	ap_memory	img	array
img_ce0	out	1	ap_memory	img	array
img_q0	in	8	ap_memory	img	array
out_r_address0	out	4	ap_memory	out_r	array
out_r_ce0	out	1	ap_memory	out_r	array
out_r_we0	out	1	ap_memory	out_r	array
out_r_d0	out	32	ap_memory	out_r	array

Рисунок 3 – Интерфейс HLS-модуля cnn_accel после синтеза

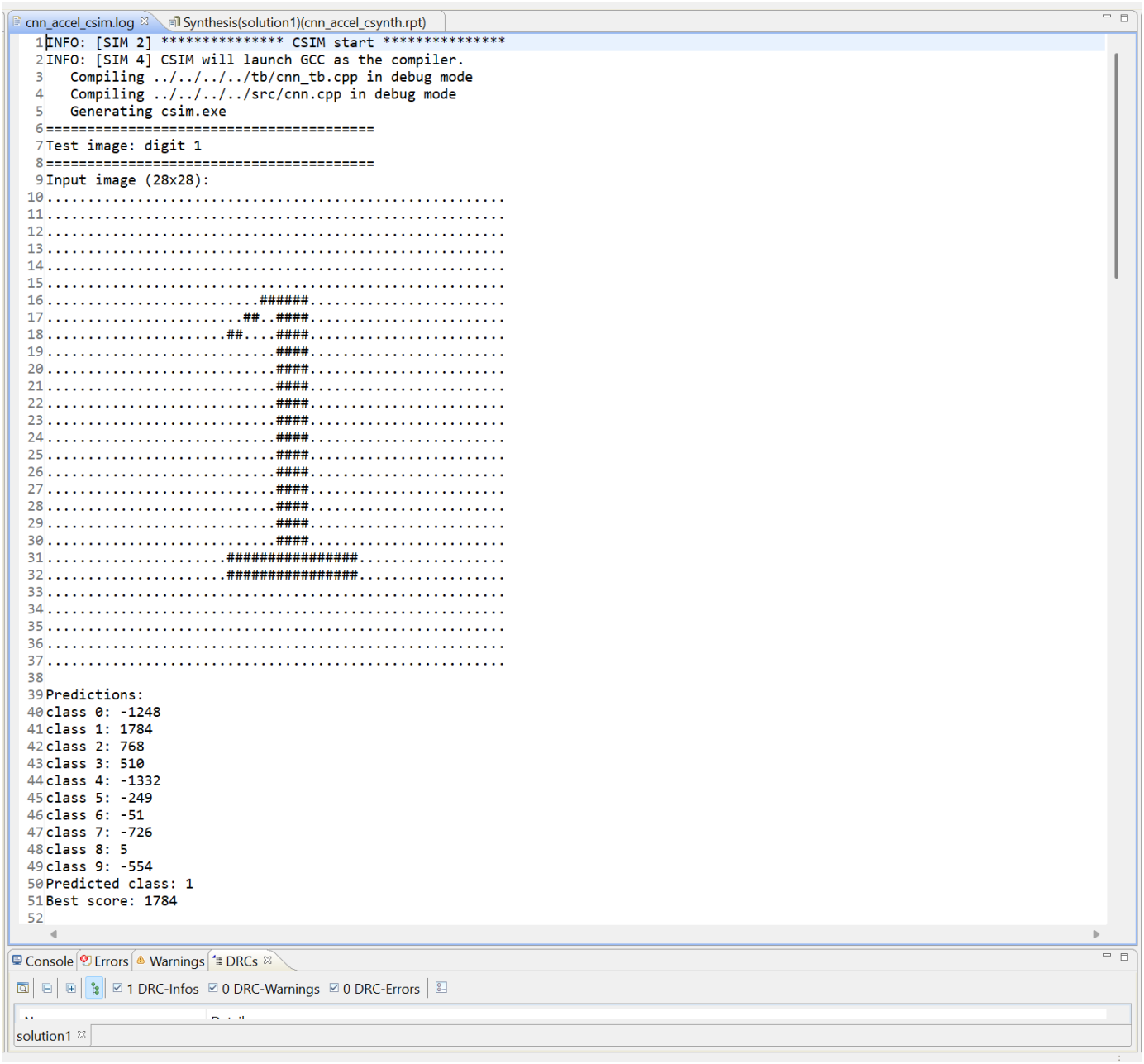


Рисунок 4 – Результат C-симуляции HLS-модели для digit 1

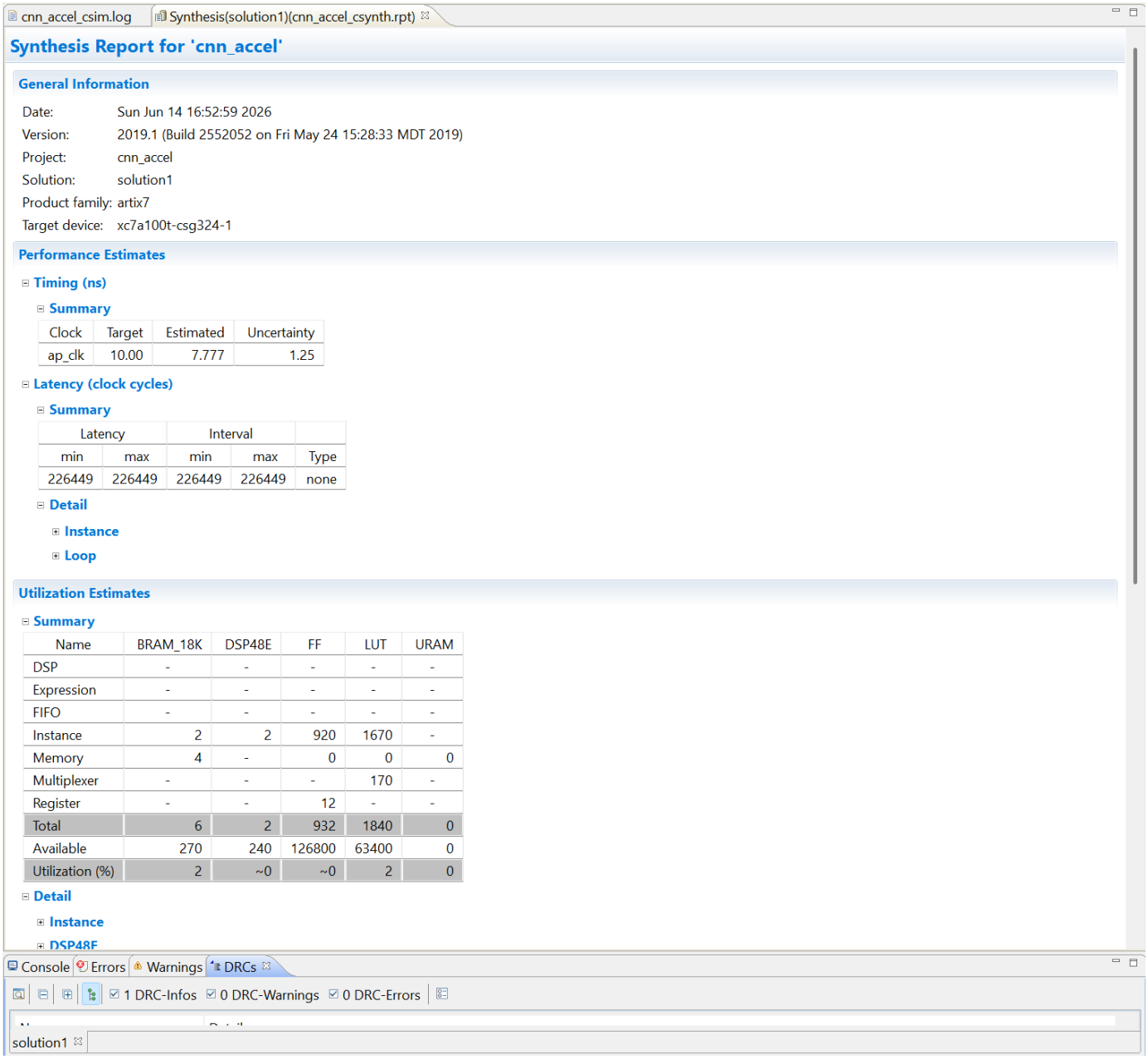


Рисунок 5 – Сводный отчет HLS-синтеза контрольной конфигурации cnn_accel

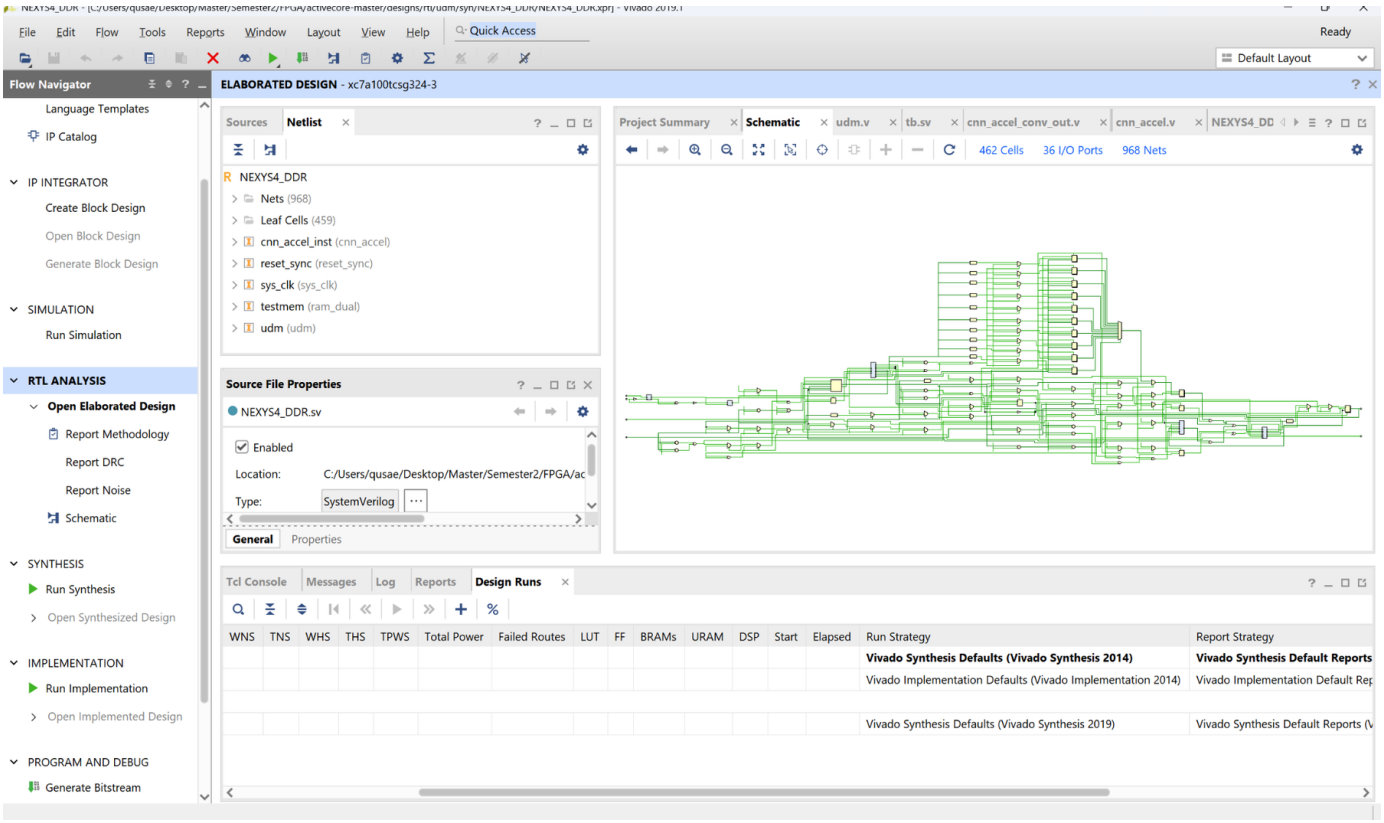


Рисунок 6 – Интегрированный проект NEXYS4_DDR в режиме Elaborated Design

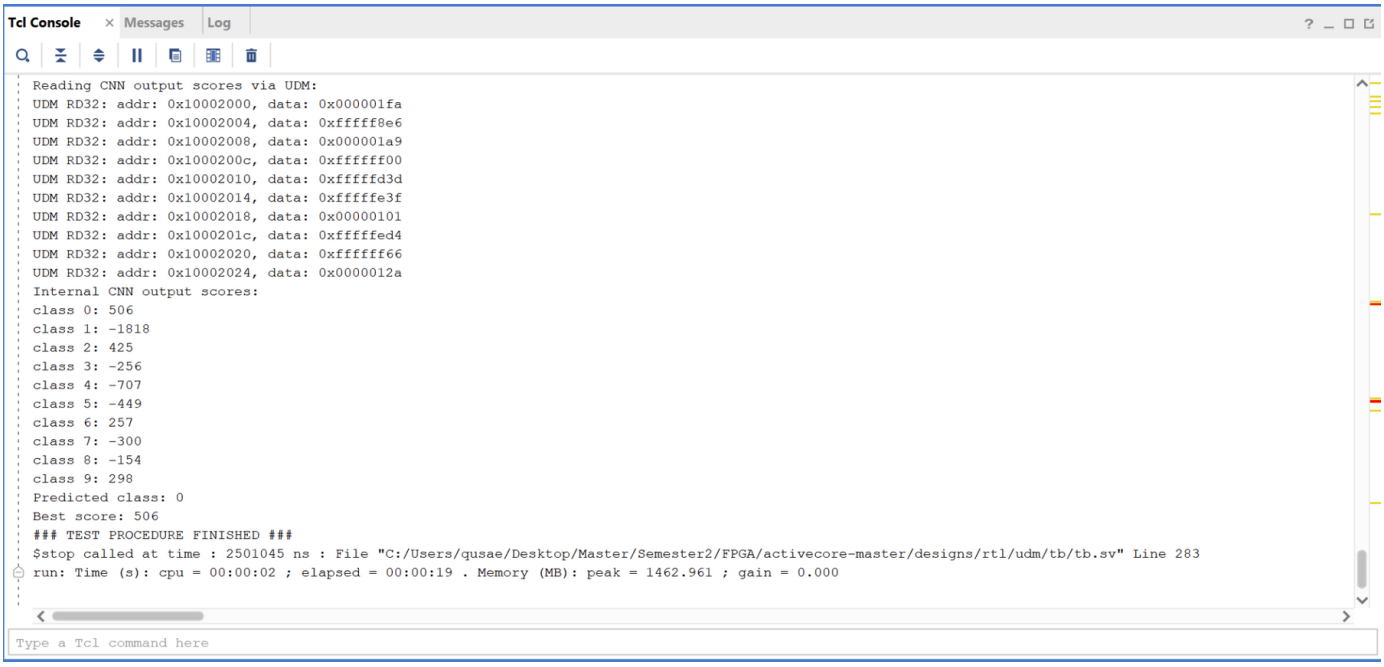


Рисунок 7 – RTL-симуляция интегрированной реализации для digit 0



Рисунок 8 – RTL-симуляция интегрированной реализации для digit 7

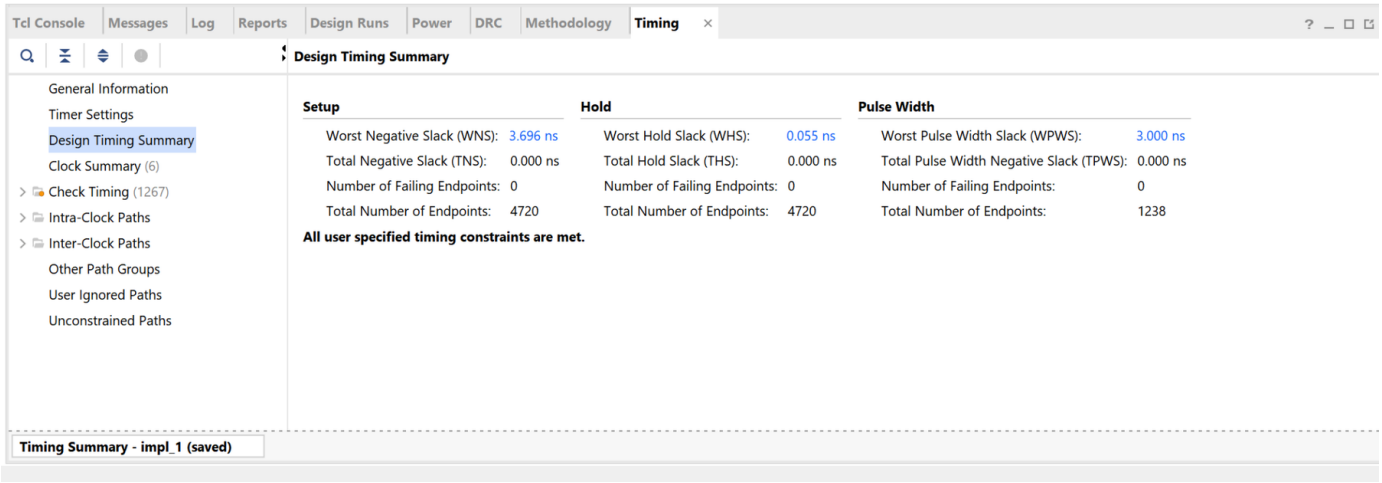


Рисунок 9 – Timing summary после implementation контрольной версии

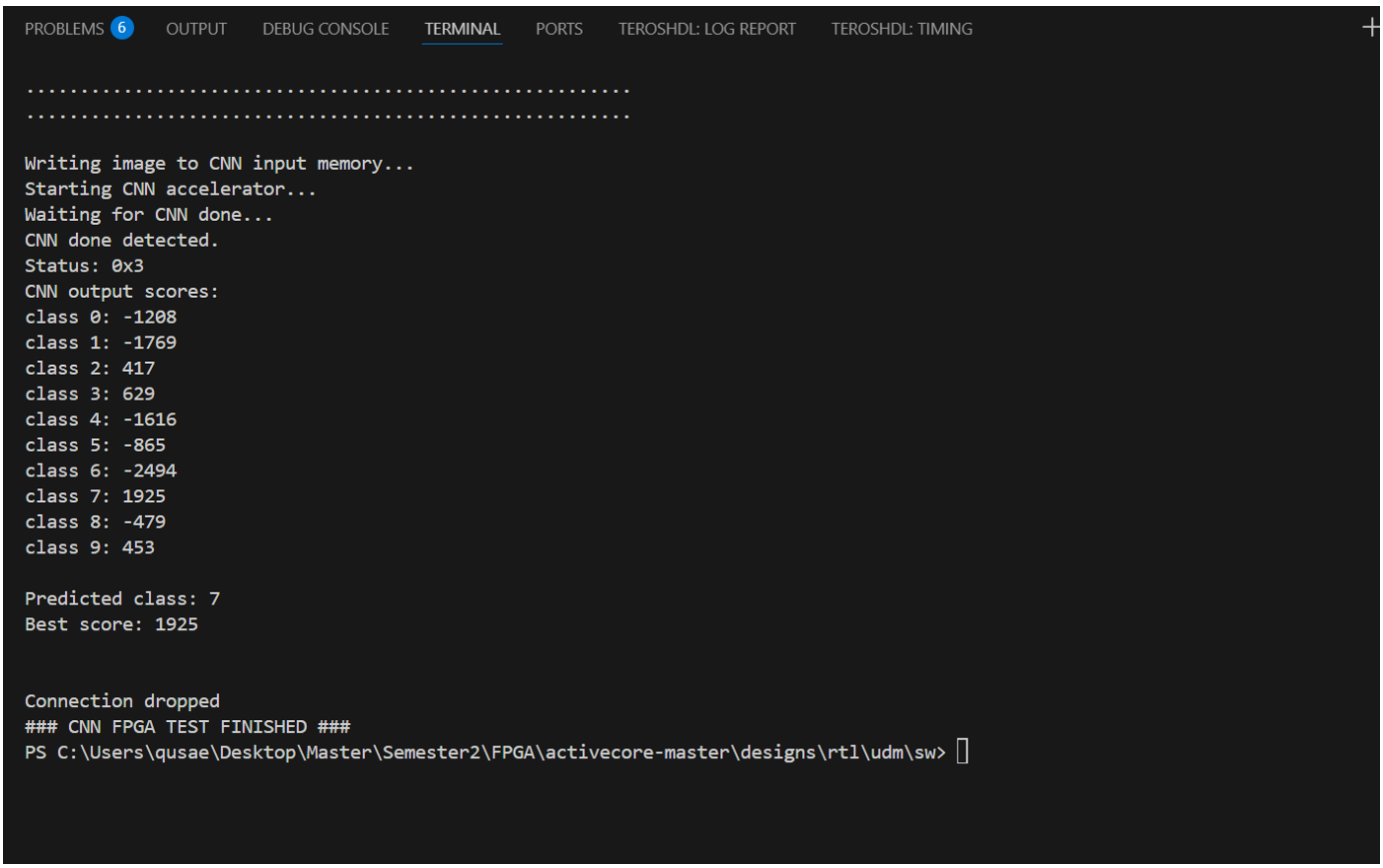


Рисунок 10 – Аппаратный запуск контрольной версии на FPGA для digit 7

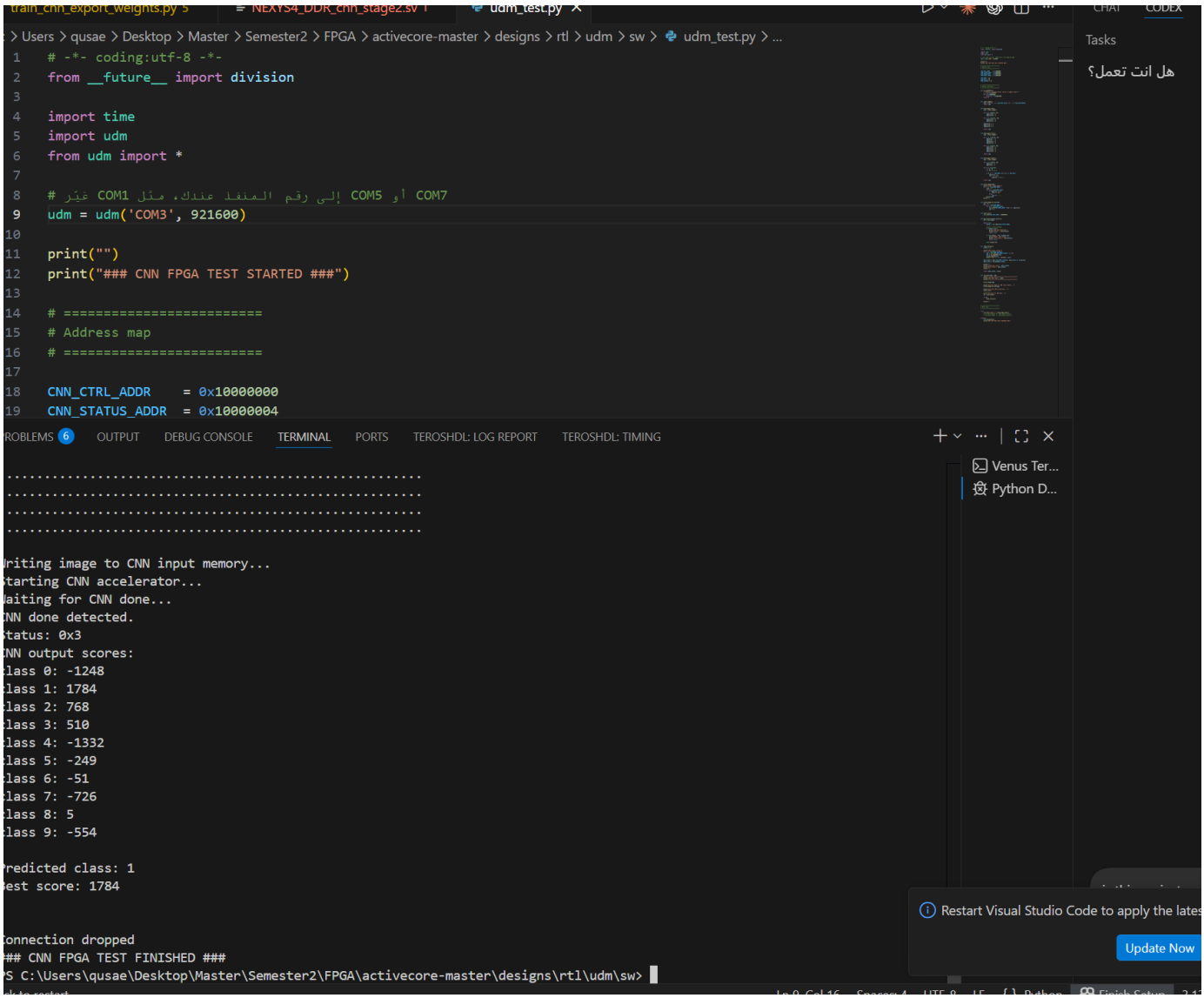


Рисунок 11 – Аппаратный запуск контрольной версии на FPGA для digit 1

ПРИЛОЖЕНИЕ Г

Материалы проверки оптимизированной версии

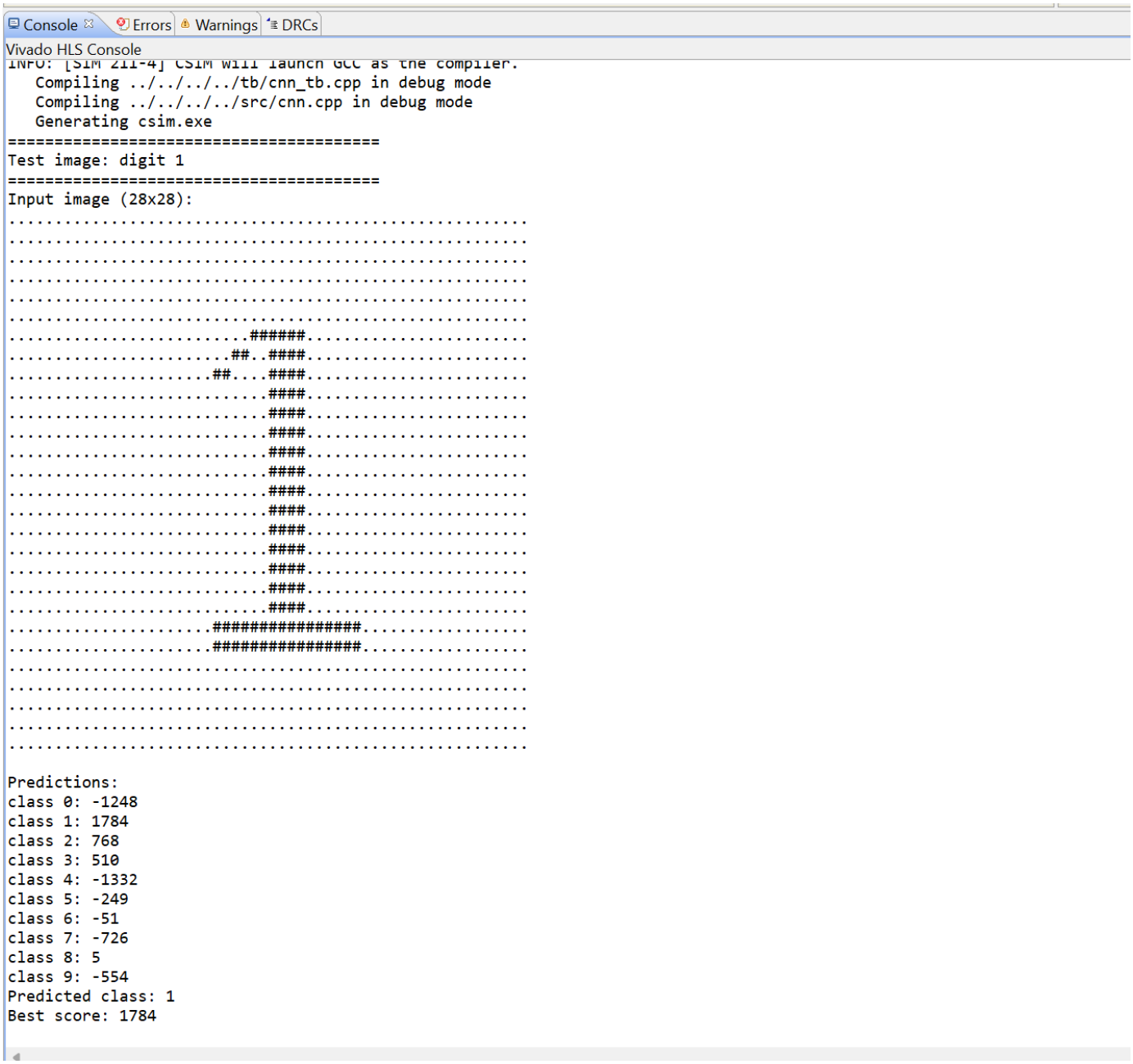
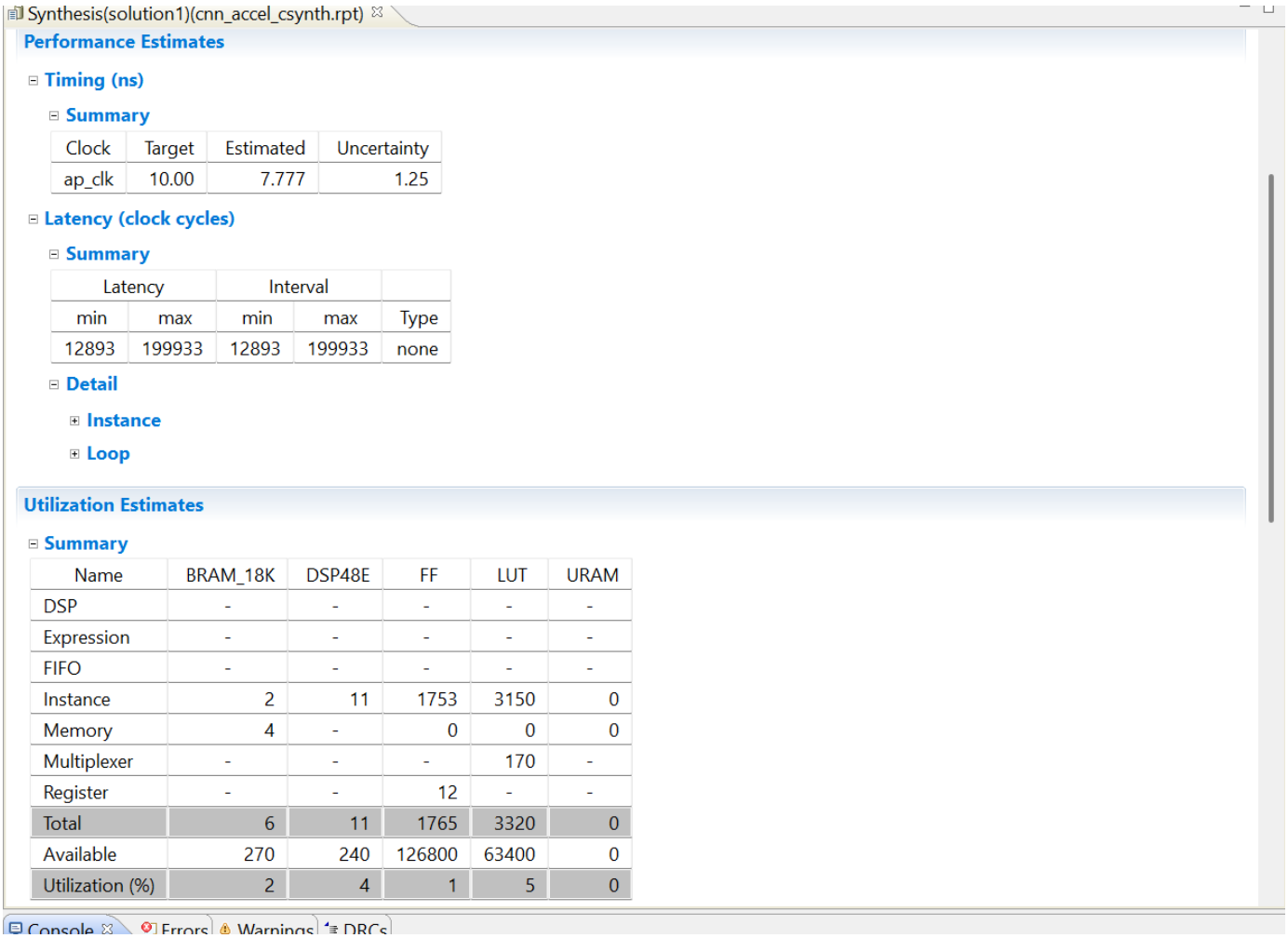


Рисунок 12 – C-симуляция оптимизированной версии для digit 1



Synthesis(solution1)(cnn_accel_csynth.rpt)

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	7.777	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
12893	199933	12893	199933	none

Detail

Instance

Loop

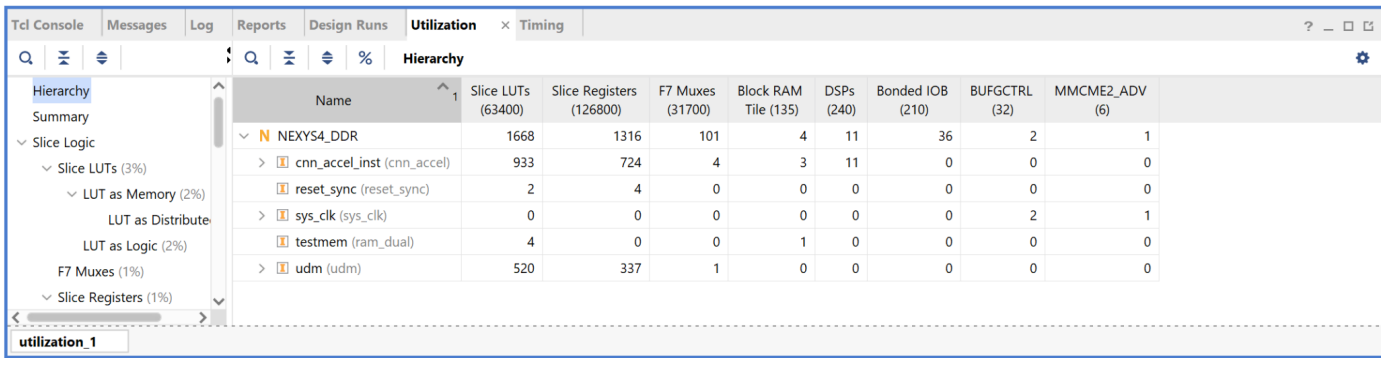
Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	-	-	-	-
Expression	-	-	-	-	-
FIFO	-	-	-	-	-
Instance	2	11	1753	3150	0
Memory	4	-	0	0	0
Multiplexer	-	-	-	170	-
Register	-	-	12	-	-
Total	6	11	1765	3320	0
Available	270	240	126800	63400	0
Utilization (%)	2	4	1	5	0

Console | Errors | Warnings | DRCs

Рисунок 13 – HLS-синтез оптимизированной версии



Tcl Console | Messages | Log | Reports | Design Runs | Utilization | Timing

Hierarchy

Hierarchy
Summary
Slice Logic
Slice LUTs (3%)
LUT as Memory (2%)
LUT as Distribute
LUT as Logic (2%)
F7 Muxes (1%)
Slice Registers (1%)

Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Block RAM Tile (135)	DSPs (240)	Bonded IOB (210)	BUFGCTRL (32)	MMCME2_ADV (6)
NEXYS4_DDR	1668	1316	101	4	11	36	2	1
cnn_accel_inst (cnn_accel)	933	724	4	3	11	0	0	0
reset_sync (reset_sync)	2	4	0	0	0	0	0	0
sys_clk (sys_clk)	0	0	0	0	0	0	2	1
testmem (ram_dual)	4	0	0	1	0	0	0	0
udm (udm)	520	337	1	0	0	0	0	0

utilization_1

Рисунок 14 – Post-implementation utilization оптимизированной версии

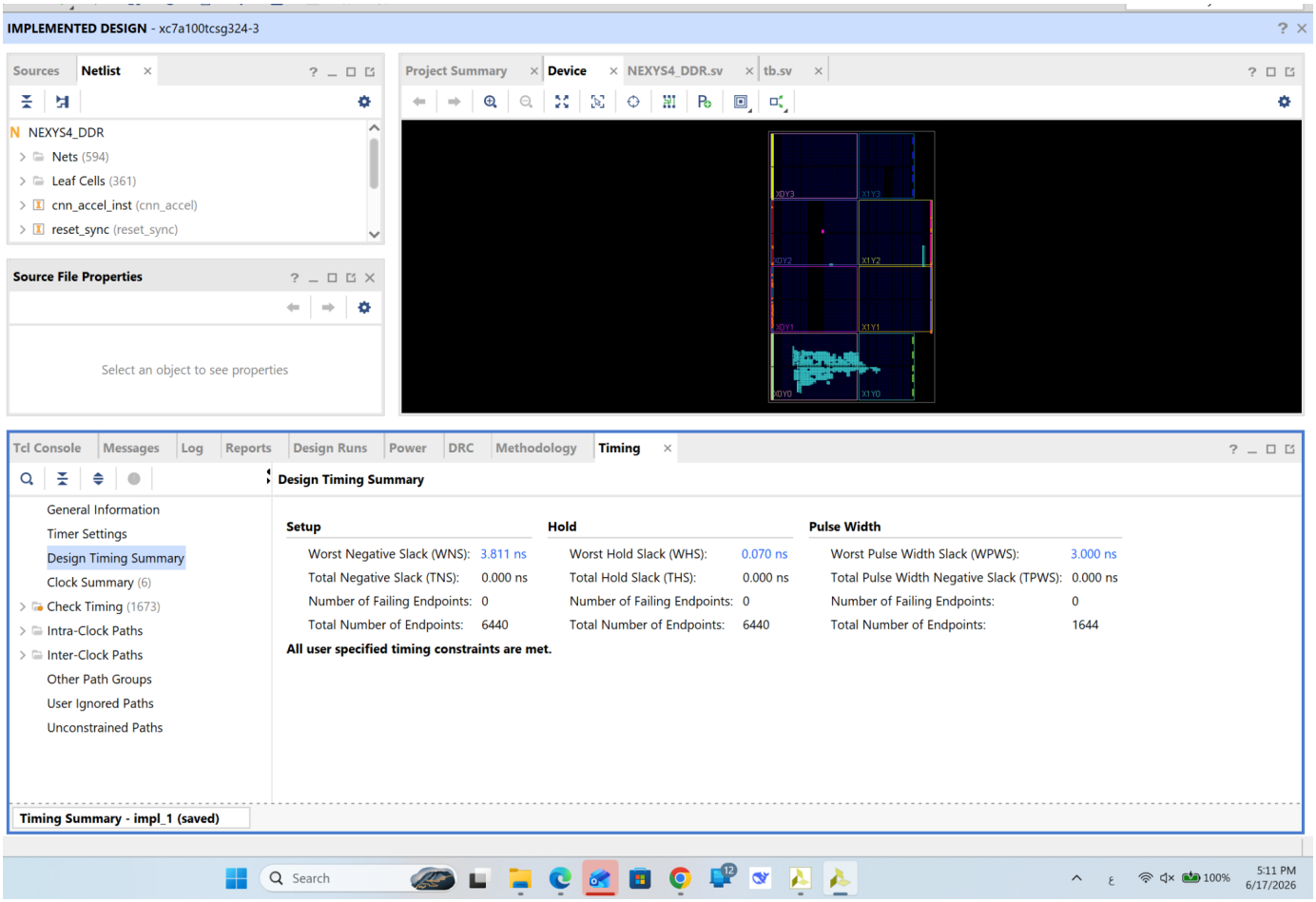


Рисунок 15 – Timing summary оптимизированной версии после implementation

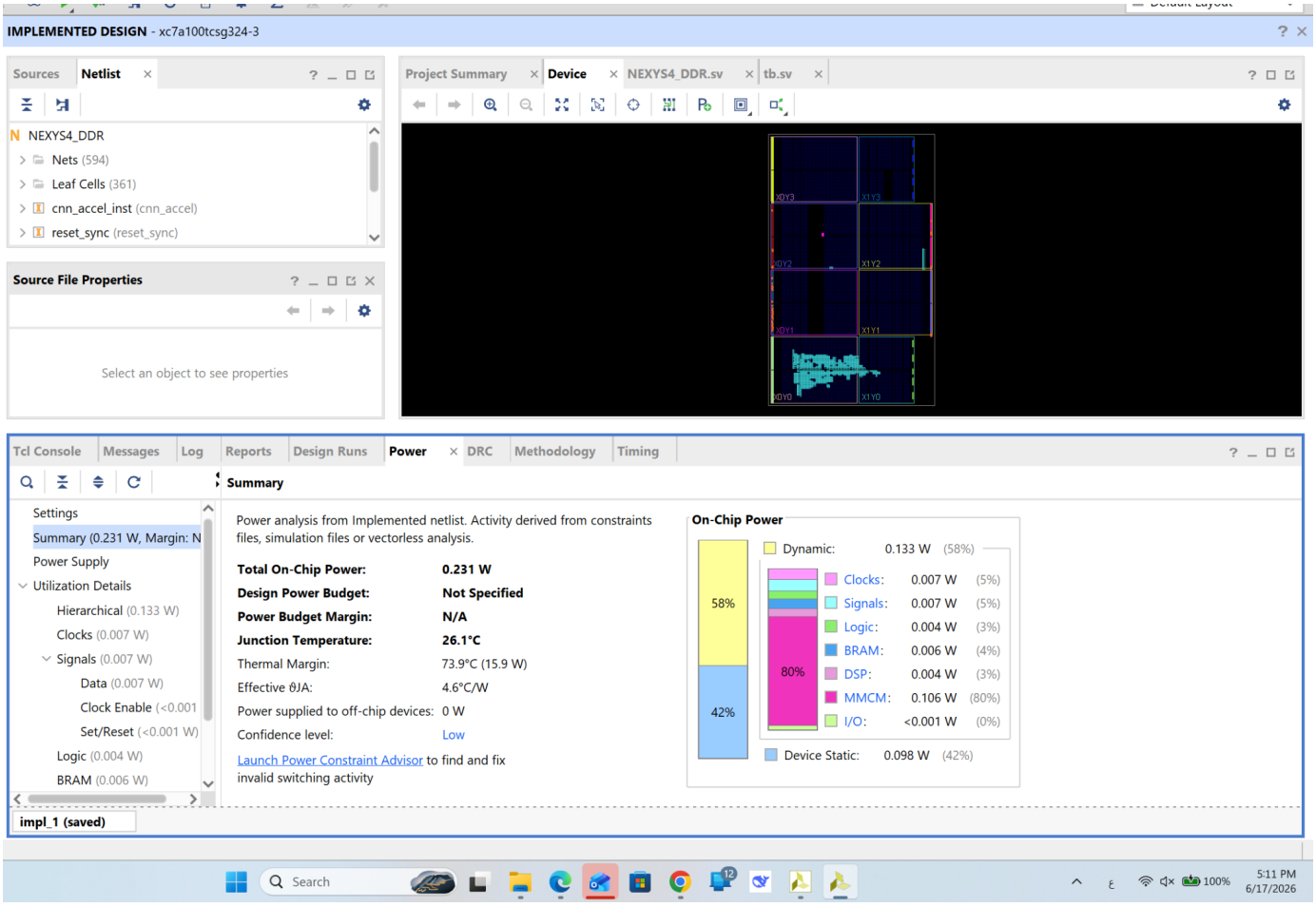


Рисунок 16 – Оценка энергопотребления оптимизированной версии

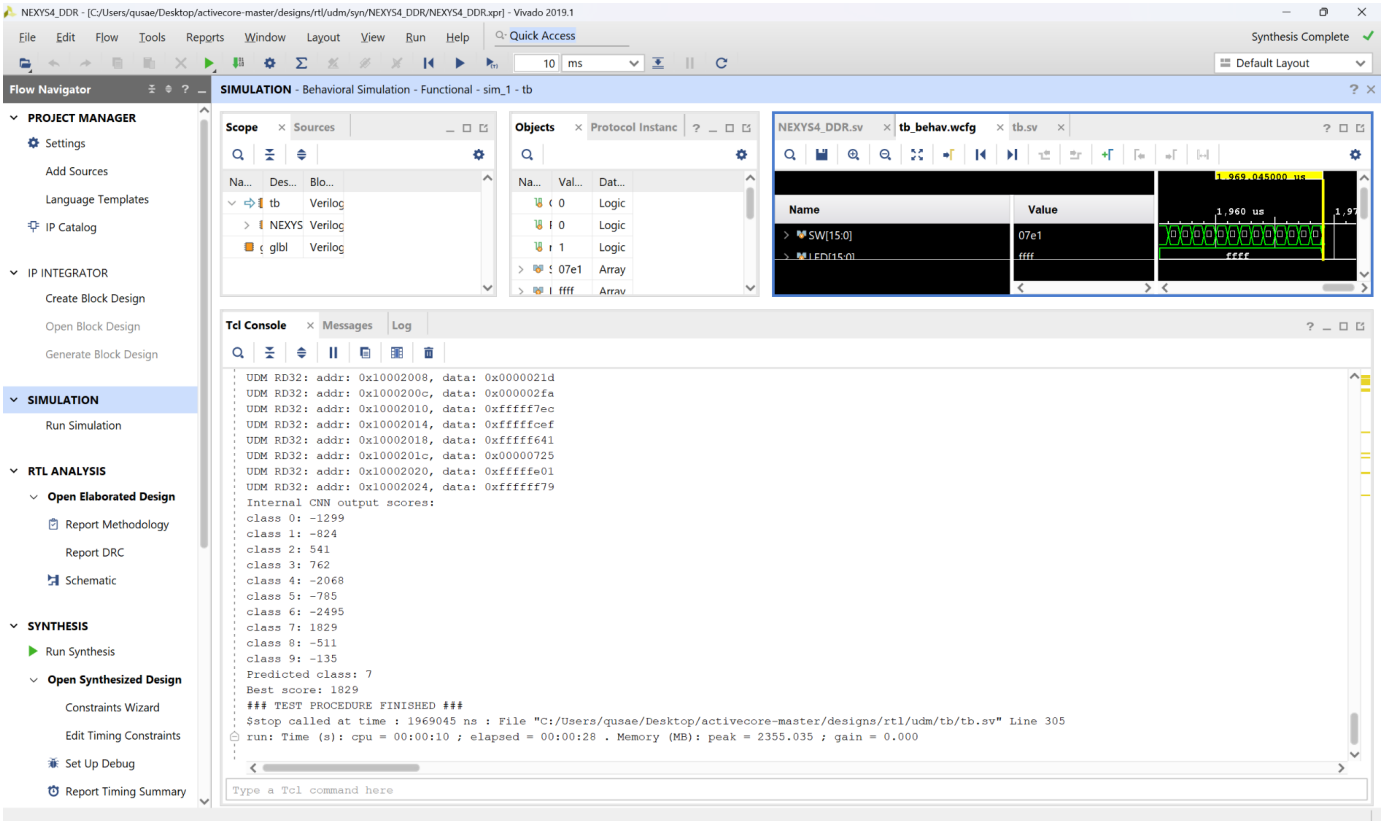


Рисунок 17 – RTL-симуляция оптимизированной версии для digit 7

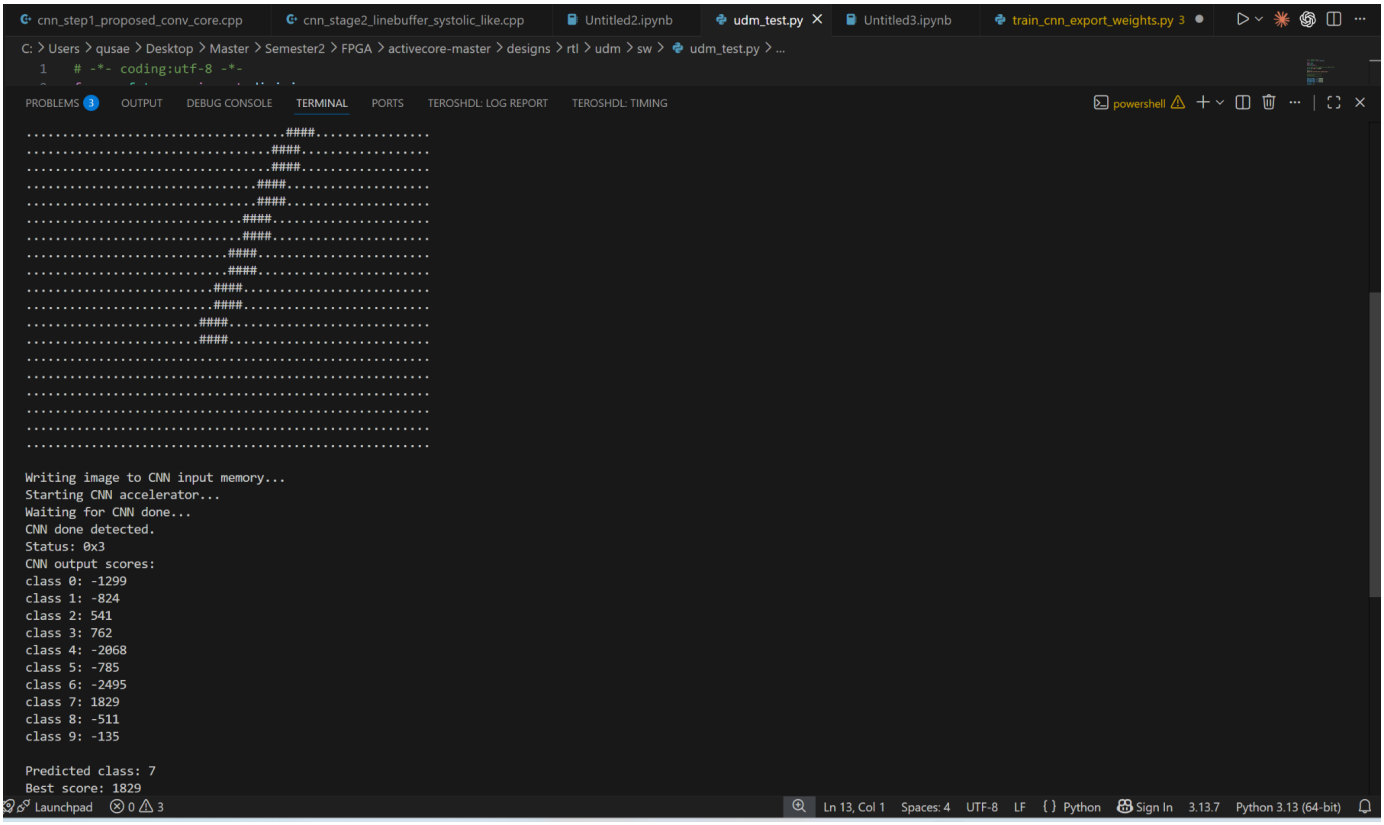


Рисунок 18 – Аппаратный запуск оптимизированной версии на FPGA для digit 7

ПРИЛОЖЕНИЕ Д

Фрагменты исходного кода

Листинг 1: Верхняя функция контрольной HLS-реализации

```
typedef int8_t  data_t;
typedef int8_t  weight_t;
typedef int32_t acc_t;

#define IMG_ROWS 28
#define IMG_COLS 28
#define KERNEL_SIZE 5
#define CONV_OUT_DIM 24
#define POOL_DIM 12
#define FILTERS 4
#define FLAT_SIZE (FILTERS * POOL_DIM * POOL_DIM)
#define NUM_CLASSES 10

void cnn_accel(
    data_t img[IMG_ROWS][IMG_COLS],
    acc_t out[NUM_CLASSES]
) {
#pragma HLS INTERFACE ap_memory port=img
#pragma HLS INTERFACE ap_memory port=out
#pragma HLS INTERFACE ap_ctrl_hs port=return

    data_t conv_out[CONV_OUT_DIM][CONV_OUT_DIM][FILTERS];
    data_t pool_out[POOL_DIM][POOL_DIM][FILTERS];
    data_t flat[FLAT_SIZE];

    conv_layer(img, conv_out);
    maxpool_layer(conv_out, pool_out);
    flatten_layer(pool_out, flat);
    fc_layer(flat, out);
}
```

Листинг 2: Фрагмент line buffer в оптимизированной версии

```
static acc_t conv5x5_linebuf_mac(
    data_t linebuf[KERNEL_SIZE][IMG_COLS],
    int top_row,
    int col,
    int f
```

```
) {
#pragma HLS INLINE
    acc_t row_psum[KERNEL_SIZE];
#pragma HLS ARRAY_PARTITION variable=row_psum complete dim=1

    for (int ki = 0; ki < KERNEL_SIZE; ki++) {
#pragma HLS UNROLL
        acc_t row_sum = 0;
        int buf_row = (top_row + ki) % KERNEL_SIZE;
        for (int kj = 0; kj < KERNEL_SIZE; kj++) {
#pragma HLS UNROLL factor=2
            row_sum += linebuf[buf_row][col + kj] * conv1_weights[f][ki][kj];
        }
        row_psum[ki] = row_sum;
    }

    acc_t sum = conv1_bias[f];
    for (int ki = 0; ki < KERNEL_SIZE; ki++) {
#pragma HLS UNROLL
        sum += row_psum[ki];
    }
    return sum;
}
```

Листинг 3: Фрагмент Python-скрипта аппаратного тестирования

```
def write_image_to_cnn(img):
    idx = 0
    for i in range(IMG_ROWS):
        for j in range(IMG_COLS):
            udm.wr32(CNN_IMAGE_BASE + idx * 4, img[i][j])
            idx += 1

def start_cnn():
    udm.wr32(CNN_CTRL_ADDR, 0x00000001)

def wait_done(timeout_sec=5.0):
    t0 = time.time()
    while True:
        status = udm.rd32(CNN_STATUS_ADDR)
        if status & 0x1:
            return True
```

```
        if time.time() - t0 > timeout_sec:
            return False
        time.sleep(0.01)
```